

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Algoritmy strojového učení pro
adaptaci autonomních agentů v 3D
akční hře

Machine Learning Algorithms for
Adapting Autonomous Agents in a
3D Action Game

DIPLOMOVÁ PRÁCE

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2025

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jochec** Jméno: **Martin** Osobní číslo: **502714**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávací katedra/ústav: **Katedra softwarového inženýrství**
Studijní program: **Aplikace informatiky v přírodních vědách**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Název diplomové práce anglicky:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Pokyny pro vypracování:

1. Nastudujte literaturu týkající se strojového učení v herních prostředích a adaptace agentů.
2. Implementujte předem navržené algoritmy a experimentálně je otestujte ve vhodném herním simulátoru.
3. Vylepšete navržené algoritmy s cílem zvýšit efektivitu adaptace agentů v dynamických prostředích.
4. Vyhodnoťte výsledky experimentů a porovnejte efektivitu jednotlivých verzí algoritmů adaptace agentů v implementovaném herním simulátoru.

Seznam doporučené literatury:

1. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge: MIT Press. ISBN 978-0262039246.
2. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Hoboken: Pearson. ISBN 978-0134610993.
3. Abbeel, P., & Schulman, J. (2016). Continuous control with deep reinforcement learning. arXiv preprint. Dostupné z: <https://arxiv.org/abs/1509.02971>.
4. Liébana, D. P., Lucas, S. M., et al. (2020). General Video Game Artificial Intelligence. Springer International Publishing. ISBN 978-3031021220.

Jméno a pracoviště vedoucí(ho) diplomové práce:

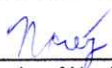
Ing. Josef Nový, Ph.D. Katedra softwarového inženýrství FJFI Děčín

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) diplomové práce:

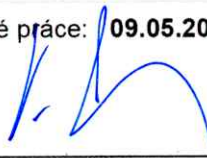
Datum zadání diplomové práce: **29.10.2024**

Termín odevzdání diplomové práce: **09.05.2025**

Platnost zadání diplomové práce: **28.10.2026**


Ing. Josef Nový, Ph.D.
podpis vedoucí(ho) práce


podpis vedoucí(ho) ústavu/katedry


doc. Ing. Václav Čuba, Ph.D.
podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Diplomant bere na vědomí, že je povinen vypracovat diplomovou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v diplomové práci.

31.10.2024
Datum převzetí zadání


Podpis studenta

PROHLÁŠENÍ

Já, níže podepsaný(á)

Příjmení, jméno studenta: Johec Martin

Osobní číslo: 502714

Název programu: Aplikace informatiky v přírodních vědách

prohlašuji, že jsem bakalářskou/diplomovou práci s názvem

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

.....

vypracoval(a) samostatně a uvedl(a) veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

☒ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil/a nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil/a. Stvrzuji, že jsem si vědom/a, že za obsah závěrečné práce plně zodpovídám.

☐ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce nepoužil/a žádný nástroj umělé inteligence. Jsem si vědom/a důsledků, kdy bude zjevné nepřiznané použití těchto nástrojů při tvorbě jakékoli části mé závěrečné práce.

v Praze dne 7.4.2025

.....
Johec
Podpis

Poděkování

Děkuji Ing. Josefu Novému, Ph.D., za vedení mé diplomové práce a za případnou pomoc při její tvorbě.

Bc. Martin Johec

Název práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Autor: Bc. Martin Jochec

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Diplomová práce

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt: Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je vytvořit jednoduchého autonomního agenta, který dokáže přizpůsobit své chování i vybavení protihráči. Práce představuje existující herní agenty, jako jsou AlphaStar, OpenAI Five a Voyager, a popisuje některé algoritmy a techniky používané při jejich vývoji – například Proximal Policy Optimization nebo Self-Play. Dále se zaměřuje na použité technologie, například Unreal Engine či NevarokML, a na samotný návrh, který je s jejich pomocí realizován. Závěrem jsou výsledky experimentu vyhodnoceny.

Klíčová slova: hry, umělá inteligence, adaptace, autonomní agent, strojové učení

Title:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Author: Bc. Martin Jochec

Abstract: This thesis focuses on machine learning algorithms for adapting an autonomous agent to user behavior in a 3D action game. The main objective is to create a simple autonomous agent capable of adjusting its behavior and equipment in response to the player. The work introduces existing game agents such as AlphaStar, OpenAI Five, and Voyager, and describes some of the algorithms and techniques used in their development – for example, Proximal Policy Optimization and Self-Play. It then focuses on the technologies used, such as Unreal Engine and NevarokML, and the design that is implemented using them. Finally, the results of the experiment are evaluated.

Key words: games, artificial intelligence, adaptation, autonomous agent, machine learning

Obsah

Úvod	9
1 AI agenti ve hrách a jejich algoritmy a techniky	11
1.1 Strojové učení a algoritmy zpětnovazebního učení	11
1.1.1 Proximal Policy Optimization (PPO)	12
1.1.2 Deep Q-Network	15
1.1.3 Self-Play	17
1.1.4 Population Based Training	19
1.2 AI agenti ve hrách	21
1.2.1 AlphaStar	21
1.2.2 OpenAI Five	24
1.2.3 Voyager	26
2 Použité technologie a návrh hry	29
2.1 Použité technologie	29
2.1.1 Unreal Engine	29
2.1.2 Visual Studio	31
2.1.3 NevarokML	32
2.2 Návrh hry	34
2.2.1 Návrh prostředí hry	34
2.2.2 Návrh agenta	35
3 Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí	37
3.1 Základní logika hry	37
3.1.1 Hráčská postava	37
3.1.2 Zbraňový systém	39
3.1.3 Logika řízení herních kol	40
3.1.4 Systém sbírání předmětů	40
3.1.5 Krabice s vybavením	41
3.1.6 Herní rozhraní (HUD)	41
3.2 Autonomní agent	42
3.2.1 První verze agenta	42
3.2.2 Druhá verze agenta	51
3.2.3 Finální verze agenta	56
3.2.4 Shrnutí všech verzí	61

Závěr	63
Literatura	64
Přílohy	68
A Jak zprovoznit program	69
B Jak opravit problém s programem	71

Úvod

Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je navrhnout jednoduchého autonomního agenta, který se pomocí strojového učení a změny vybavení dokáže přizpůsobit protivníkovi a porazit ho.

První kapitola obsahuje dvě podkapitoly. První se věnuje vybraným technikám a algoritmům strojového učení, jako jsou Proximal Policy Optimization (PPO), Deep Q-Network (DQN), a techniky Self-Play a Population Based Training (PBT). Druhá podkapitola popisuje existující herní agenty, konkrétně AlphaStar, OpenAI Five a Voyager, včetně přehledu jejich fungování a způsobu implementace.

Druhá kapitola se zaměřuje na použité technologie a návrh samotné hry. První podkapitola podrobně popisuje využití Unreal Engine, Visual Studio a pluginu Nevarok ML, který rozšiřuje Unreal Engine o funkcionalitu pro trénink a aplikaci modelů strojového učení. Druhá podkapitola se věnuje návrhu herního prostředí a autonomního agenta.

Třetí kapitola popisuje implementaci hry a agenta. První část se zaměřuje na herní logiku, která poskytuje prostředí pro aplikování agenta. Druhá část se věnuje implementaci samotné logiky agenta, pokusům o vylepšení jeho logiky a porovnání různých verzí.

Závěr shrnuje úvahy autora a dosažené výsledky — zda se agent adaptuje podle očekávání, jaký vliv mají jednotlivá vylepšení a jak si různé verze modelu vedou ve vzájemném porovnání.

Kapitola 1

AI agenti ve hrách a jejich algoritmy a techniky

1.1 Strojové učení a algoritmy zpětnovazebního učení

Strojové učení je oblast umělé inteligence zaměřená na vývoj algoritmů, které umožňují systémům zlepšovat své chování na základě dat bez nutnosti explicitního naprogramování konkrétních pravidel. Tyto algoritmy se učí rozpoznávat vzory a rozhodovací strategie přímo z datových vstupů a adaptují se podle získaných zkušeností.

Využití strojového učení je obzvláště důležité v případech, kdy nelze snadno vytvořit ručně definovaná pravidla — například při řízení autonomních agentů v nelineárním či stochastickém prostředí.

Existují čtyři hlavní typy strojového učení:

1. **Učení s učitelem (Supervised Learning)** – modely se učí na základě dat s předem známým výstupem.
2. **Učení bez učitele (Unsupervised Learning)** – cílem je nalézt strukturu nebo vzory v datech bez označených výstupů.
3. **Částečně řízené učení (Semi-Supervised Learning)** – kombinuje malé množství označených dat s velkým množstvím neoznačených.
4. **Zpětnovazební učení (Reinforcement Learning)** – agent se učí prostřednictvím interakcí s prostředím a získává odměny za dosažení cílů.

Z hlediska této práce je nejdůležitější čtvrtý přístup – zpětnovazební učení (RL), které umožňuje agentům učit se strategie řízení chování prostřednictvím zpětné vazby získané z prostředí. Následující kapitoly se proto zaměřují právě na RL a konkrétní algoritmy, které tento přístup využívají.

1.1.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)[7][9] je jedním z nejrozšířenějších algoritmů pro trénování agentů pomocí hlubokého zpětnovazebního učení. Vyvinut byl výzkumníky ze společnosti OpenAI jako reakce na klíčové nedostatky tehdejších metod policy gradient algoritmů, které často provádějí příliš velké aktualizace politiky bez jakékoli kontroly, a složitější metody jako Trust Region Policy Optimization (TRPO)[8], které jsou výpočetně náročné a obtížně implementovatelné. PPO nabízí kompromis: spojuje jednoduchost implementace s vysokou stabilitou učení a efektivním využitím dat.

Algoritmus PPO patří do rodiny metod založených na optimalizaci politiky. Na rozdíl od metod hodnotové funkce, jako je Q-learning[13][21], přímo modeluje politiku agenta (pravděpodobnostní rozdělení nad akcemi) a učí se ji optimalizovat tak, aby maximalizoval dlouhodobou očekávanou odměnu. PPO navazuje na TRPO, který zavádí omezení velikosti aktualizace politiky, aby se zabránilo destruktivním skokům ve strategii agenta, ale činí tak výrazně jednodušším a výpočetně efektivnějším způsobem.

Základní myšlenka PPO spočívá v tom, že při každé aktualizaci politiky se zamezuje příliš velkým změnám pravděpodobnostních rozdělení nad akcemi. Konkrétně se zavádí tzv. *clipped surrogate objective*, která upravuje klasický výraz pro policy gradient, aby penalizovala změny mimo stanovený rozsah. Tento přístup umožňuje provádět více kroků stochastického gradientního vzestupu na stejných datech bez rizika destabilizace učení.

Matematicky lze optimalizovaný výraz popsat jako:

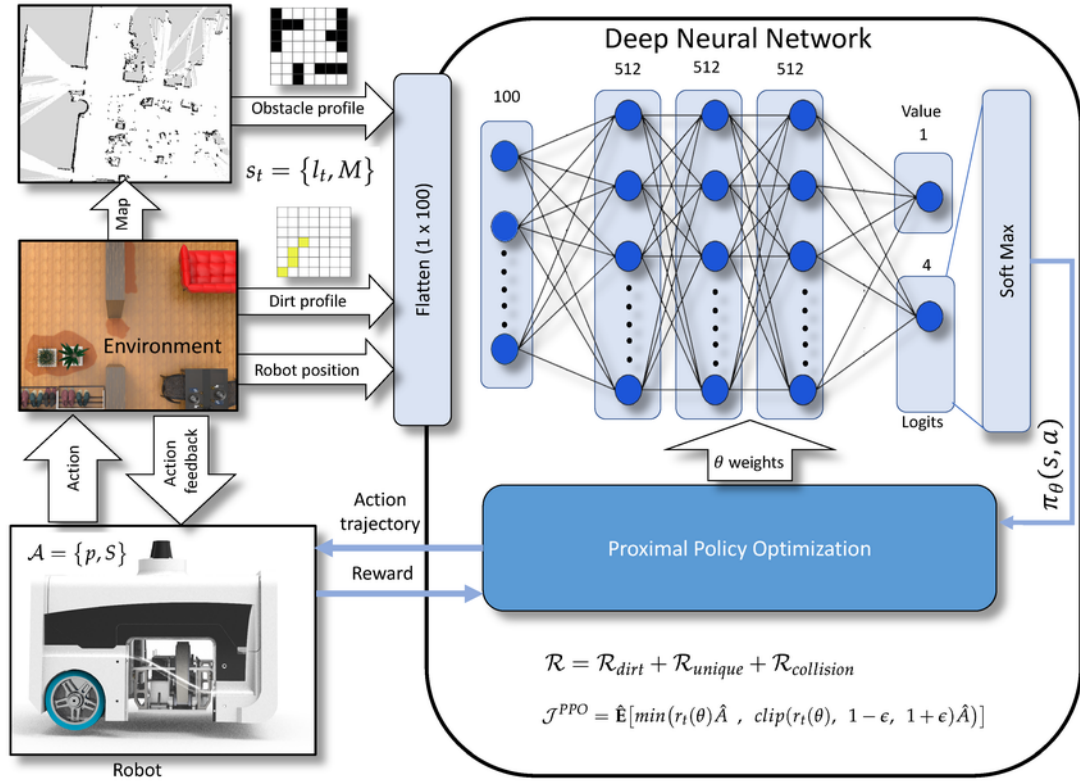
$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

kde

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Je poměr pravděpodobností podle nové a staré politiky a $\hat{A}_t$ je odhad tzv. advantage funkce, která kvantifikuje, jak výhodná je daná akce vzhledem k očekávanému vývoji. Hodnota ϵ je hyperparametr, typicky nastavený na 0,1 až 0,3, který určuje toleranci změn politiky. Symbol $\hat{\mathbb{E}}_t$ představuje empirické očekávání vypočtené na základě vzorků nasbíraných během interakcí agenta s prostředím.

Smyslem výrazu *clip* je omezit motivaci agenta k tomu, aby příliš radikálně měnil svou strategii, pokud taková změna vede k výraznému zvýšení pravděpodobnosti akcí. Pokud je změna politiky příliš výrazná (např. $r_t > 1 + \epsilon$), je výpočet výsledku „oříznut“, aby nevedl k nadhodnocení zisku. Naopak, pokud je změna v rámci povoleného pásma, výpočet se ponechává. Výsledkem je stabilnější učení, které se vyhýbá oscilacím a přetížení aktualizací. Celý proces vizuálně shrnuje obrázek 1.1, který zachycuje architekturu PPO včetně interakce s prostředím, výpočtu hodnoty akce a aktualizace politiky.



Obrázek 1.1: Vizualizace architektury algoritmu PPO. Agent je ovlivňován prostředím, zatímco dvě neuronové sítě — actor a critic — slouží k výběru akcí a odhadu hodnoty stavu. Politika se aktualizuje pomocí oříznutého objektivu (clipped objective) a výhody se počítají pomocí metody Generalized Advantage Estimation (GAE).[27]

Kromě této hlavní ztrátové funkce lze algoritmus PPO dále rozšířit o dvě další složky — penalizaci chyby odhadu hodnoty a entropickou regulaci. Celková ztrátová funkce pak má následující tvar:

$$L_t^{PPO}(\theta) = \mathbb{E}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 \mathcal{S}(\pi_\theta, s_t)] ,$$

Kde L_t^{VF} představuje kvadratickou chybu hodnotové funkce, $\mathcal{S}(\pi_\theta, s_t)$ označuje entropii distribuční politiky ve stavu s_t a c_1, c_2 jsou váhové koeficienty. Přidání entropické složky pomáhá zajistit průzkum prostředí (exploration), zejména v počátečních fázích tréninku, a zabraňuje tomu, aby se agent „zasekl“ v lokálním maximu.

Pro výpočet výše zmíněné advantage funkce $\hat{A}_t$ se často používá metoda Generalized Advantage Estimation (GAE)[10]. Tato metoda snižuje rozptyl odhadů tím, že kombinuje okamžité odměny a odhady hodnoty pomocí exponenciálního vážení parametrizovaného faktorem λ :

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots,$$

kde

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

a γ představuje diskontní faktor. GAE tak umožňuje lepší kompromis mezi zkreslením a rozptylem, což se v praxi promítá do rychlejšího a stabilnějšího učení.

Implementace PPO obvykle probíhá ve stylu tzv. actor–critic. Paralelní agenti (herní instance) generují trajektorie, obvykle o délce stovek časových kroků, a tyto vzorky se následně využívají v několika iteracích minibatch SGD nad ztrátovou funkcí. V takové architektuře actor reprezentuje politiku agenta a volí akce, zatímco critic odhaduje hodnotu stavu a dodává zpětnou vazbu, která pomáhá zpřesňovat politiku.

Při tréninku se standardně používá optimalizátor Adam a struktura s oddělenými parametry politiky a hodnotové funkce. V některých případech lze ale využít i sdílené parametry mezi těmito částmi.

Z praktického hlediska se PPO úspěšně aplikuje na širokou škálu prostředí. V oblasti spojitě kontroly (např. úkoly v prostředí MuJoCo[5]) překonává jiné běžně používané přístupy jako TRPO, A2C nebo CEM, a to jak z hlediska dosažené odměny, tak z hlediska rychlosti konvergence. Rovněž ve hrách z prostředí Atari, kde se porovnává s A2C a ACER, vykazuje PPO vysokou robustnost napříč různými herními scénáři. Oproti metodám s pevným KL penalizačním koeficientem dosahuje stabilnějších výsledků díky použití tzv. adaptive KL penalty nebo oříznutí podle výše uvedeného pravidla.

PPO se stává preferovaným algoritmem v mnoha aplikacích nejen díky svému výkonu, ale i díky snadné integraci do různých tréninkových rámců. Kromě empirických výsledků spočívá jeho výhoda i v jednoduchosti. Z hlediska implementace vyžaduje pouze mírnou úpravu klasického policy gradient frameworku a nevyžaduje řešení složitých optimalizačních úloh jako TRPO (např. pomocí konjugovaného gradientu). Díky tomu se PPO řadí mezi standardní volby v mnoha open-source knihovnách zpětnovazebního učení, včetně OpenAI Baselines, Stable Baselines nebo RLlib.

Celkově lze PPO vnímat jako rozumný kompromis mezi výpočetní náročností, robustností a výkonem. Jeho úspěch spočívá v jednoduché, ale účinné úpravě klasických policy gradient metod, která zajišťuje, že každá aktualizace politiky zůstává „blízko“ té původní, čímž se zamezuje katastrofálním změnám a zároveň se umožňuje efektivní učení.

1.1.2 Deep Q-Network

Algoritmus Deep Q-Network (DQN)[21][22][23] představuje metodu zpětnovazebního učení, která kombinuje klasický Q-learning s hlubokými neuronovými sítěmi pro aproximaci akčně-hodnotové funkce $Q(s, a)$. Funkce $Q(s, a)$ udává očekávanou kumulovanou diskontovanou odměnu po provedení akce a ve stavu s a následném sledování optimální politiky. Teoretický základ DQN tvoří Bellmanova optimální rovnice:

$$Q^*(s_t, a_t) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t, a_t \right]$$

kde s_t označuje stav, a_t zvolenou akci, r_{t+1} okamžitou odměnu, s_{t+1} následující stav a $\gamma \in [0, 1]$ diskontní faktor. Klasický Q-learning aktualizuje tabulku hodnot podle vztahu:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

kde α označuje rychlost učení. V prostředích s vysokodimenzionálním stavovým prostorem není tabulkové ukládání hodnot možné. DQN využívá hlubokou neuronovou síť $Q(s, a; \theta)$ s parametry θ , která na vstupu přijímá stav s a na výstupu poskytuje vektor Q-hodnot pro všechny akce.

Trénink DQN minimalizuje časový rozdíl (TD error) mezi odhadovanou Q-hodnotou a cílovou hodnotou z Bellmanovy rovnice. Stabilizace učení využívá dvě hlavní techniky: přehrávací paměť (experience replay) a cílovou síť (target network). Přehrávací paměť $\mathcal{D}$ ukládá přechody $(s_t, a_t, r_{t+1}, s_{t+1})$ a umožňuje náhodné vzorkování mini-batchů, čímž přerušuje korelace a zvyšuje rozmanitost tréninkových dat. Cílová síť $Q(s, a; \theta^-)$ představuje kopii online sítě, která se aktualizuje periodicky, aby poskytovala stabilní cílové hodnoty.

Pro přechod v mini-batchi se cílová hodnota y_t a ztrátová funkce $L_t(\theta)$ definují následovně:

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$$

$$L_t(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(y_t - Q(s_t, a_t; \theta))^2 \right]$$

Parametry θ se aktualizují stochastickým gradientním sestupem:

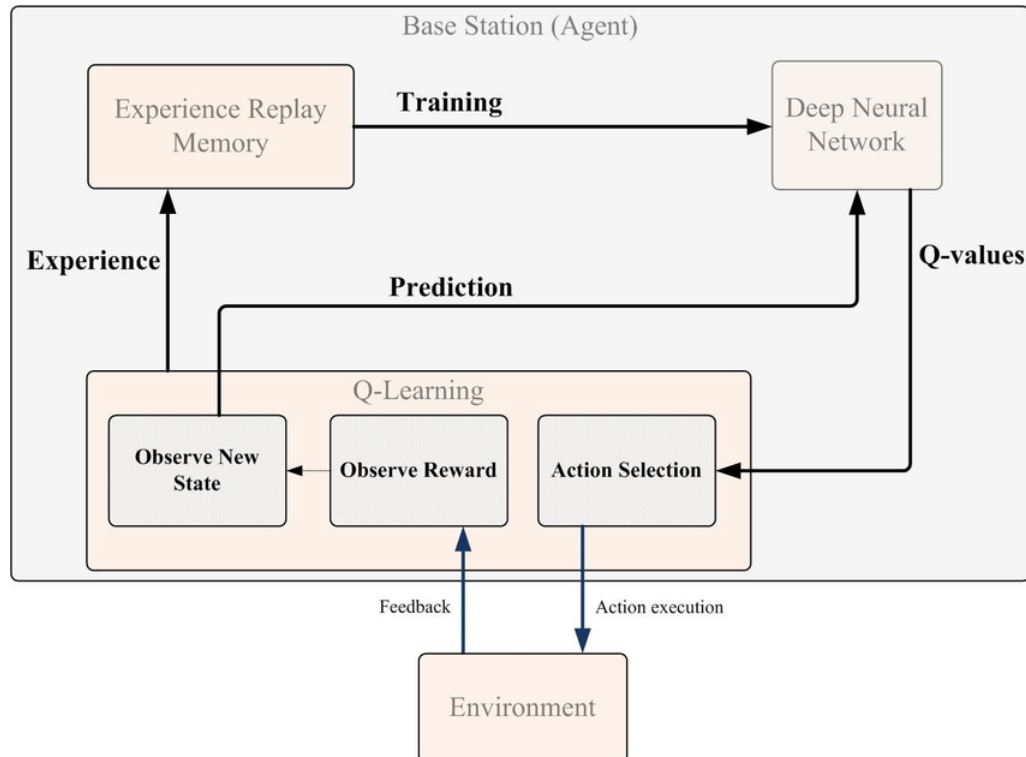
$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_t(\theta)$$

Váhy cílové sítě se synchronizují s hlavní sítí periodicky:

$$\theta^- \leftarrow \theta$$

Tréninkový cyklus zahrnuje sledování stavu s_t , výběr akce a_t pomocí ϵ -greedy politiky, provedení akce, získání odměny r_{t+1} a nového stavu s_{t+1} , uložení přechodu do paměti $\mathcal{D}$, vzorkování mini-batche, výpočet y_t , minimalizaci TD chyby a periodickou aktualizaci cílové sítě.

Algoritmus DQN je citlivý na nestabilitu a divergenci kvůli kombinaci nelineární aproximace neuronovou sítí a bootstrapovaných aktualizací. Přehrávací paměť redukuje autokorelaci dat a zvyšuje efektivitu využití zkušeností, cílová síť poskytuje stabilní referenční hodnoty. Pokročilé varianty, jako jsou Double DQN, Dueling DQN a Prioritized Experience Replay, zlepšují konvergenci, stabilitu a efektivitu učení v komplexních prostředích.



Obrázek 1.2: Příklad architektury Deep Q-Learning[30]

1.1.3 Self-Play

Self-play[16] je technika v oblasti zpětnovazebního učení, která umožňuje agentům zlepšovat své strategie prostřednictvím interakce se sebou samými — tedy s kopiemi, staršími verzemi nebo současnými protivníky v rámci jednoho tréninkového rámce. Tato metoda se ukazuje jako zásadní pro dosažení vyšších výkonů ve hrách, jako jsou Go, šachy, StarCraft II nebo Dota 2. Na rozdíl od klasického zpětnovazebního učení, kde agent interaguje s předem daným prostředím, zde prostředí zahrnuje další agentní entity, jejichž chování se v čase mění — a tím vytváří dynamický a nelineární tréninkový proces.

Základní přístup self-play spočívá v tom, že agent trénuje proti svým vlastním kopiím, ať už současným, nebo historickým. Nejjednodušší forma, tzv. „vanilla self-play“, znamená, že se agent neustále střetává s poslední verzí sebe sama. Tato metoda funguje velmi efektivně v tranzitivních hrách, kde lze jednotlivé strategie jednoznačně seřadit dle síly. V nestranově tranzitivních prostředích (například kámen-nůžky-papír) však často vede ke cyklickému chování a může konvergovat k suboptimálním strategiím.

Aby se tyto nedostatky zmírnily, vznikají složitější varianty. „Fiktivní self-play“ rozšiřuje trénink tak, že agent hraje proti průměru všech svých předchozích verzí. Tím se posiluje robustnost učení a eliminuje riziko zacyklení. „Prioritizovaná fiktivní self-play“ (např. použitá v AlphaStar) dále zavádí vážený výběr protivníků, čímž zvýhodňuje ty, proti nimž si agent vede hůře — tedy záměrně trénuje proti „slabým místům“ své strategie.

Další variantou je „ δ -uniform self-play“, která používá hyperparametr δ k určení, jaké části historické populace se použijí jako soupeři. Nižší δ znamená širší spektrum historických politik, vyšší hodnoty naopak preferují nedávné soupeře.

Moderní rámce self-play algoritmů se často integrují s tzv. „population-based“ přístupy. Místo jednoho agenta existuje populace politik Π , která se v čase rozšiřuje o nové verze. Každá nová politika se trénuje vůči vzorku soupeřů vybíraných z Π na základě určité metastrategie Σ , často reprezentované interakční maticí. Tento proces vytváří bohatý prostor pro evoluci strategií a vzájemnou adaptaci.

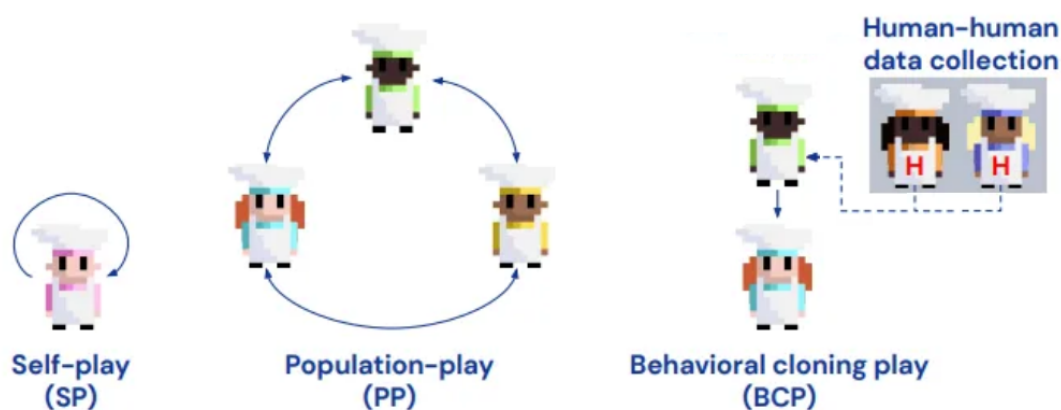
Základní tréninková smyčka obvykle probíhá následovně:

1. Nové politiky jsou inicializovány, často na základě posledních iterací předchozích politik.
2. Každá politika je trénována proti soupeřům vybraným podle Σ .
3. Nově natrénované politiky se přidají do populace a proces se opakuje.

Z hlediska výpočetní efektivity rozlišujeme mezi tzv. „lazy initialization“, kdy se politiky inicializují až v okamžiku potřeby, a „immediate initialization“, kdy se celá populace vytváří hned na začátku. Dále rozlišujeme „standardní trénink“, kde se v každé iteraci trénuje pouze jeden agent, a „průběžný trénink“ (např. FTW agent), kde se všechny politiky trénují souběžně.

Zvláštní pozornost si zasluhuje tzv. PSRO (Policy-Space Response Oracles), které rozšiřují klasické self-play o výpočet rovnovážných strategií v rámci tzv. meta-herního prostoru. Místo toho, aby se agent učil proti jednotlivým politikám, PSRO používá například Nashovu rovnováhu jako metastrategii, vůči níž se hledají nejlepší odpovědi.

Kromě klasických her nachází self-play široké uplatnění i v komplexních videohrách a reálných scénářích, například v robotice, autonomním řízení nebo syntéze proteinů. Také se, zatím v menší míře, využívá v kontextu velkých jazykových modelů — například pro sebezdokonalování (self-improvement) nebo samostatnou konstrukci výukových dat. Základní rozdíl mezi přístupy self-play, population-play a behavioral cloning play ilustruje obrázek 1.3.



Obrázek 1.3: Srovnání tří přístupů k tréninku agentů: „self-play“ (SP), kde agent hraje sám proti sobě; „population-play“ (PP), kde interaguje s různými agenty v populaci; a „behavioral cloning play“ (BCP), kde se učení zakládá na lidských demonstračních datech. Upraveno podle zdroje[28]

1.1.4 Population Based Training

Population-Based Training (PBT)[4] je technika, která propojuje samotný trénink neuronových sítí s automatickým laděním hyperparametrů. Místo toho, aby se model trénuje s pevně danou konfigurací a po dokončení se hledají optimální nastavení v dalších bězích, PBT tento proces spojuje — optimalizuje váhy i hyperparametry zároveň a průběžně. To je zvláště užitečné v oblastech, kde se optimální hyperparametry v čase mění, například u zpětnovazebního učení nebo generativních modelů.

Na začátku PBT inicializuje populaci modelů, z nichž každý má své vlastní váhy i hyperparametry. Všichni členové této populace se trénují paralelně a nezávisle — často metodami jako gradient descent. Po určitém počtu kroků, nebo jakmile model dosáhne stanoveného výkonu, nastává tzv. „exploit-and-explore“ fáze. Pokud některý z modelů vykazuje podprůměrný výkon, jeho obsah se nahrazuje jiným, výkonnějším modelem z populace. Zároveň se na něj aplikuje lehce pozměněná verze jeho hyperparametrů — čímž vzniká nový „směr“ učení.

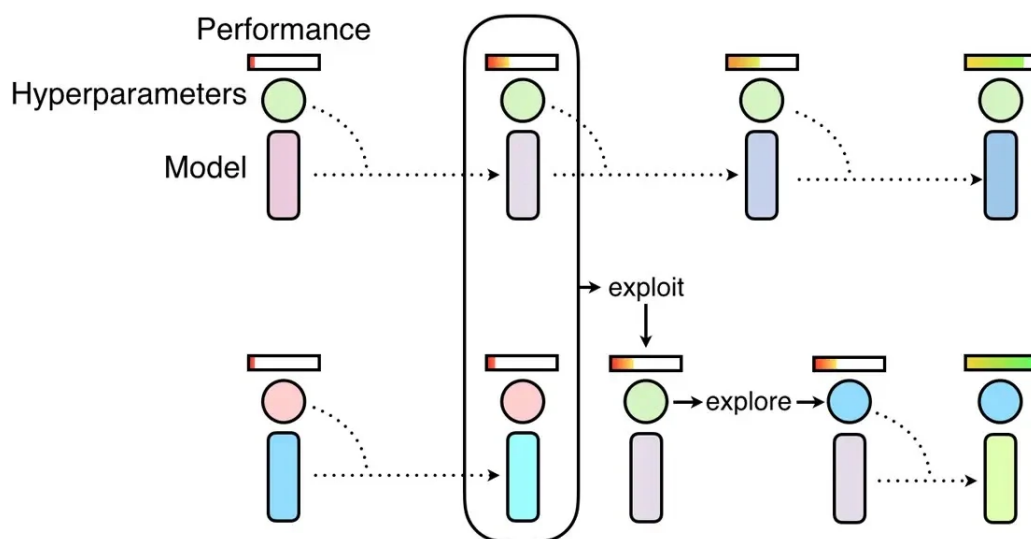
Tento přístup vytváří smyčku, ve které probíhá klasický trénink modelů, ale průběžně se mezi nimi také sdílejí informace a testují nové kombinace. Tím dochází nejen k rychlejší konvergenci, ale také ke zlepšení finálního výkonu, protože hyperparametry nejsou fixní, ale průběžně se přizpůsobují aktuálnímu stavu tréninku.

Z praktického hlediska je PBT velmi dobře škálovatelný. Nevyžaduje centrální koordinaci — modely komunikují pouze prostřednictvím jednoduchého sdílení výkonných metrik a checkpointů. Díky tomu lze PBT provozovat i na heterogenních nebo nestabilních výpočetních platformách, například v cloudovém prostředí s preemptivními instancemi.

Základní smyčka algoritmu vypadá následovně:

1. Každý model ve své instanci trénuje pomocí standardních optimalizačních metod (například SGD nebo Adam) s aktuální konfigurací hyperparametrů.
2. Po určité době se vyhodnocuje jeho výkon (například pomocí validace nebo herní odměny).
3. Pokud je model označen jako „ready“, probíhá fáze „exploit“ — model si kopíruje váhy a hyperparametry od výkonnějšího modelu v populaci.
4. Následuje fáze „explore“, při které se hyperparametry lehce pozměňují (například násobkem 1,2 nebo 0,8), případně se náhodně přegenerovávají.
5. Trénink pokračuje s nově nastavenými parametry.

Tento proces ilustruje obrázek 1.4, který ukazuje, jak si modely mezi sebou sdílejí váhy a mění hyperparametry na základě výkonnosti.



Obrázek 1.4: Population-Based Training neuronových sítí začíná jako náhodné prohledávání, ale v průběhu tréninku umožňuje jednotlivým modelům využít dílčí výsledky ostatních (fáze „exploit“) a zároveň zkoušet nové kombinace hyperparametrů (fáze „explore“). [29]

Důležité je, že PBT není citlivý na přesné načasování fáze „exploit-and-explore“ — obvykle stačí, aby mezi dvěma takovými událostmi uplynulo dost kroků na to, aby gradientové učení mělo čas projevit efekt.

Technika PBT nachází uplatnění v různých oblastech. V rámci zpětnovazebního učení například zrychluje trénink agentů v prostředích jako DeepMind Lab, StarCraft II nebo Atari. V těchto případech se automaticky ladí parametry jako learning rate, entropická penalizace nebo délka rozvinutí sítě (unroll length). Ve strojovém překladu pomocí Transformerů PBT nalézá hyperparametrové rozvrhy, které překonávají ručně laděné baseline modely. U GANů pak zlepšuje stabilitu tréninku i finální kvalitu generovaných obrázků.

Klíčovou výhodou PBT oproti klasickým metodám ladění (jako je grid search nebo Bayesian optimization) je, že probíhá „online“ — tedy současně s tréninkem. Díky tomu je výrazně efektivnější, protože modely není nutné trénovat znovu od začátku po každé změně hyperparametrů. PBT tak nabízí kombinaci výhod klasického evolučního přístupu a gradientového učení.

Celkově lze PBT vnímat jako techniku, která do optimalizace vnáší adaptivitu a sdílení znalostí v reálném čase. Výborně se doplňuje s dalšími technikami, jako je self-play.

1.2 AI agenti ve hrách

1.2.1 AlphaStar

AlphaStar[11] představuje agenta umělé inteligence (AI), kterého vyvíjí společnost DeepMind pro hraní StarCraft II[15], známé a dynamické online strategie v reálném čase od společnosti Blizzard Entertainment. Projekt je veřejnosti představen od ledna 2019. Již v srpnu téhož roku dosahuje AlphaStar hodnoty „Grandmaster“, což představuje významný milník nejen v oblasti AI zaměřené na videohry, ale do jisté míry i pro celý obor vývoje inteligentních systémů.

Hry navržené pro lidské hráče slouží jako cenná měřítko pokroku v oblasti umělé inteligence, jelikož nabízejí komplexní a dynamické prostředí s externí validitou. Zatímco dřívější milníky, jako Deep Blue[14] od IBM nebo AlphaGo od DeepMind[1][21], demonstrují schopnosti AI v tahových hrách s úplnou informací, StarCraft přináší nové výzvy díky hraní v reálném čase, částečné pozorovatelnosti, absenci jediné optimální strategie a rozsáhlému prostoru možných akcí. Přesto výzkumná komunita považuje tuto hru za řešitelnou s využitím moderních technologií, a to díky jejímu diskrétnímu charakteru, známým pravidlům a omezenému počtu objektů. To z ní činí ideální testovací prostředí pro agenty, jako je AlphaStar.

Společnost DeepMind Technologies vzniká ve Spojeném království v roce 2010. Již v roce 2011 označuje spoluzakladatel Demis Hassabis StarCraft za „další krok vpřed“ po hrách jako Go. Poté, co DeepMind demonstruje schopnosti svých agentů na základě zpětnovazebního učení (Reinforcement Learning), kteří překonávají lidské hráče v klasických hrách na Atari 2600, získává v roce 2014 podporu od společnosti Google, což otevírá cestu k dalším ambiciózním projektům — včetně vývoje AI pro StarCraft II, o který výzkumný zájem postupně roste.

Počítačový vědec Zachary Mason v roce 2015 předpovídá, že výzkum DeepMind může během pěti až deseti let vést k průlomu právě v této hře. Myšlenka získává větší váhu v roce 2016, kdy AlphaGo poráží světového šampiona v Go, Lee Sedola. Po tomto úspěchu Hassabis veřejně projevuje zájem vytvořit autonomního agenta pro StarCraft, přičemž zdůrazňuje jeho strategickou hloubku, částečnou pozorovatelnost, skryté informace a vlastnosti, které z něj činí ještě náročnější hru než Go.

Formální spolupráce mezi DeepMind a Blizzardem se oznamuje na BlizzConu v listopadu 2016 spolu s plánem na vydání otevřeného vývojového prostředí pro boty na začátku roku 2017. Do poloviny roku 2017 už DeepMind zpracovává herní data ze StarCraft II. V srpnu téhož roku pak obě společnosti uvolňují nástroje pro vývoj botů a soubor 65 000 záznamů z profesionálních zápasů. Odborníci v té době odhadují, že potrvá ještě několik let, než AI porazí špičkového hráče. Tyto předpovědi se však ukazují jako příliš opatrné, protože v prosinci 2018 poráží agent od DeepMind profesionálního hráče Grzegorze „MaNu“ Komincze poměrem 5:0. V lednu 2019 je tento agent oficiálně představen veřejnosti pod jménem „AlphaStar“. Navzdory tomuto úspěchu se však objevuje i kritika.

AlphaStar má výhodu v podobě přímé komunikace s hrou přes API, což mu umožňuje provádět akce s chirurgickou přesností. Dále disponuje globálním přehledem o mapě místo omezení herní kamerou a může v rámci daného časového okna rozkládat akce nerovnoměrně, čímž vyvíjí extrémní tlak v klíčových momentech. DeepMind na tyto připomínky reaguje přeškolením AlphaStar pod realističtějšími omezeními, která více odpovídají lidským možnostem. V následném zápase proti MaNově tato verze prohrává.

Od července 2019 soutěží upravený AlphaStar anonymně na veřejném evropském žebříčku v režimu 1v1, kde se utká s hráči, kteří s tímto experimentem souhlasí. Na konci srpna 2019, jak již bylo zmíněno, dosahuje AlphaStar hodnosti „Grandmaster“, čímž se zařazuje mezi nejlepších 0,2 % lidských hráčů a potvrzuje tím významný pokrok v oblasti autonomních agentů pro komplexní hry v reálném čase.

Zatímco zpětná propagace (Backpropagation, BP) zůstává hlavní metodou trénování neuronových sítí, AlphaStar využívá pokročilejší přístup zvaný Population-Based Training (PBT)[4]. Tento přístup kombinuje výhody evolučních algoritmů (EAs), které slouží k průzkumu globálního prostoru řešení, s lokální optimalizací pomocí BP. AlphaStar konkrétně používá lamarckovskou evoluci, kdy úspěšné sítě předávají své parametry a upravené hyperparametry méně výkonným sítím.

Pro zvládnutí vysoké výpočetní náročnosti je PBT navržen jako asynchronní a distribuovaný systém, který umožňuje paralelní trénink agentů bez nutnosti synchronizace. Na rozdíl od klasických generačních evolučních algoritmů využívá PBT tzv. steady-state přístup, kdy se agenti průběžně aktualizují bez přerušení procesu. Tento mechanismus zvyšuje efektivitu, udržuje diverzitu a umožňuje průběžné přizpůsobování hyperparametrů — což je klíčové v prostředích s proměnlivým tréninkovým chováním, jako je hluboké zpětnovazební učení.

AlphaStar dále využívá kompetitivní koevoluci, při níž se agenti neučí pouze proti svým předchozím verzím, ale proti celé populaci různorodých protivníků. Tento přístup poskytuje přirozenou tréninkovou křivku a zároveň zvyšuje robustnost strategií, jelikož agent musí zvládnout široké spektrum herních stylů. Výběr soupeřů probíhá na základě podobnosti výkonnosti (např. Elo skóre), což umožňuje plynulý rozvoj bez stagnace.

Proces učení a rozhodování AlphaStaru ilustruje konkrétní příklad herní situace, v níž agent zpracovává aktuální stav mapy, aktivuje neuronové sítě a vyhodnocuje dostupné možnosti. Vizualizaci tohoto procesu zachycuje obrázek 1.5.



Obrázek 1.5: Vizualizace rozhodovacího procesu AlphaStaru během zápasu proti hráči. Systém zpracovává syrová pozorování, aktivuje neuronové sítě, vyhodnocuje potenciální akce a předpovídá výsledky zápasu.[24]

Protože StarCraft II neobsahuje jednu univerzální vítěznou strategii, AlphaStar neusiluje o vytvoření jediného dokonalého agenta. Místo toho udržuje populaci různorodých strategií, vybraných na základě Nashovy distribuce — tedy takových, které jsou vůči sobě navzájem nejméně zranitelné.

Tento přístup odpovídá principům kvalitní diverzity (Quality Diversity, QD). Namísto hledání jediné nejlepší varianty podporují QD algoritmy vznik více kvalitních, ale odlišných řešení. Ve AlphaStaru se strategie určují pomocí herně specifických charakteristik — například preferencí pro určité jednotky nebo schopností porazit konkrétní protivníky. Tyto charakteristiky se navíc během tréninku dynamicky upravují, čímž vzniká rozmanitější a odolnější sada řešení.

Díky tomuto přístupu AlphaStar operuje s portfoliem strategií, které jsou přizpůsobivé, méně náchylné k porážce a lépe si poradí s různorodostí lidských protivníků.

1.2.2 OpenAI Five

OpenAI Five[6] představuje agenta umělé inteligence určeného pro hraní týmové videohry Dota 2, ve které proti sobě nastupují dvě pětičlenná družstva. Poprvé se veřejnosti představuje v roce 2017 při živé demonstraci, během níž poráží profesionálního hráče Dendiho v duelu jeden na jednoho. V následujícím roce systém pokročuje natolik, že zvládá ovládat celý tým pěti agentů a poráží i profesionální lidské týmy.

Společnost OpenAI volí Dota 2[3] kvůli její složitosti, nepředvídatelnosti a plynulému průběhu, který lépe odráží podmínky skutečného světa. Díky tomu slouží tato hra jako vhodné prostředí pro vývoj univerzálnějších systémů pro řešení komplexních problémů. Algoritmy a kód vytvořené pro OpenAI Five se později využívají také v dalších projektech — například pro ovládání robotické ruky pomocí neuronové sítě. OpenAI Five bývá často zmiňován vedle jiných přelomových AI systémů, které porázejí lidi v konkrétních úlohách — například AlphaStar ve StarCraft II, AlphaGo ve hře Go nebo Deep Blue v šachu.

Vývoj algoritmů, na nichž OpenAI Five stojí, začíná v listopadu 2016. OpenAI volí Dota 2 jako testovací prostředí mimo jiné kvůli popularitě hry na platformách jako Twitch, nativní podpoře Linuxu a dostupnosti aplikačního rozhraní (API).

První významná veřejná demonstrace probíhá v srpnu 2017 na turnaji The International, což je každoroční vrcholná událost ve světě Dota 2. Během této akce poráží raný model bota profesionálního hráče Dendiho v duelu jeden na jednoho. Podle technického ředitele OpenAI Grega Brockmana se systém učí metodou zpětnovazebního učení, přičemž své schopnosti zlepšuje hraním sám proti sobě po dobu dvou týdnů. V tomto režimu agent získává odměny za klíčové akce ve hře, jako je zabíjení nepřátel nebo ničení struktur.

V červnu 2018 se systém rozšiřuje do plnohodnotného pětičlenného týmu, který dokáže porážet amatérské a poloprofesionální lidské soupeře. Na The International 2018 se OpenAI Five účastní dvou exhibičních zápasů proti týmům na profesionální úrovni — proti brazilskému paiN Gaming a proti výběru bývalých elitních čínských hráčů. Přestože boti obě utkání prohrávají, zkušenost je pro vývojový tým cenná, protože umožňuje další ladění strategií a rozhodovacích mechanismů.

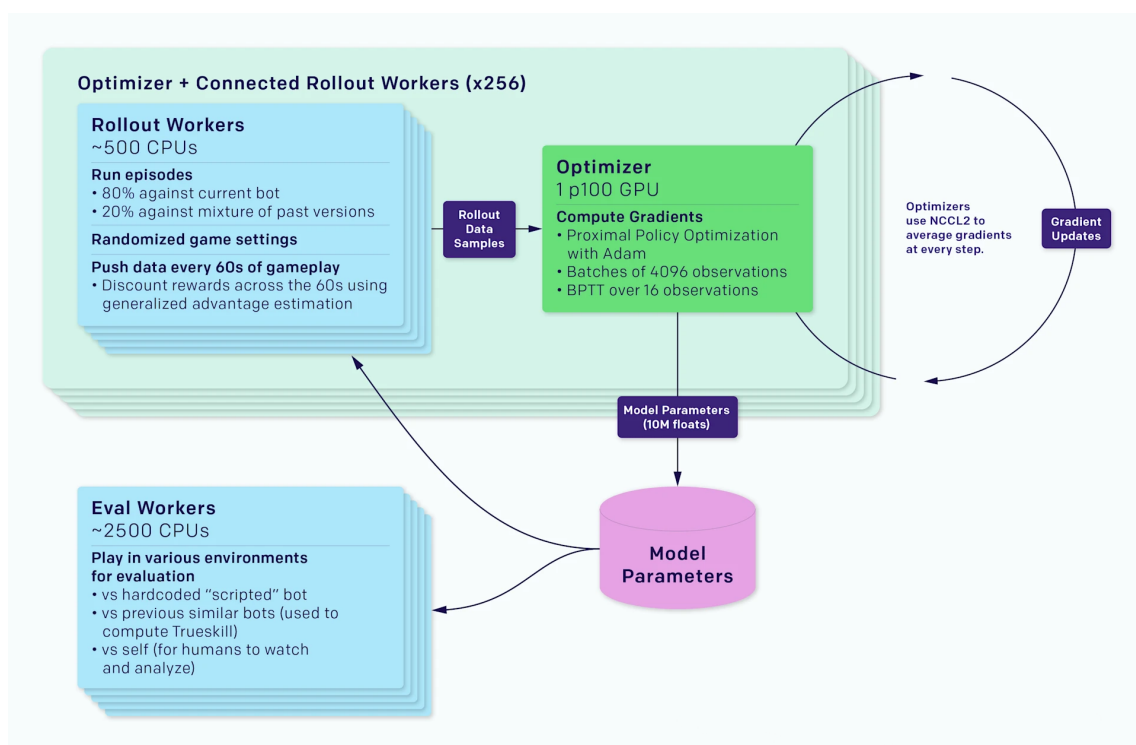
Závěrečná veřejná prezentace projektu probíhá v dubnu 2019, kdy OpenAI Five poráží v sérii na dvě vítězství tým OG, tehdejší mistry světa z turnaje The International 2018. Ve stejném měsíci probíhá i čtyřdenní online událost otevřená veřejnosti, během níž se systém účastní více než 42,000 zápasů — z nichž vyhrává přibližně 99,4

OpenAI Five je založen na rámci hlubokého zpětnovazebního učení, jehož jádrem je metoda Proximal Policy Optimization (PPO)[7][9] — gradientní algoritmus politiky, který vyvažuje stabilitu učení, jednoduchost implementace a efektivitu využití dat. PPO představuje vylepšení oproti dřívějším metodám, jako jsou běžné policy gradients nebo Trust Region Policy Optimization (TRPO), a to díky zavedení tzv. „clipped surrogate“ objektivní funkce, která zabraňuje příliš razantním aktualizacím politiky a zajišťuje konzistentní pokrok při tréninku.

Každý agent je reprezentován neuronovou sítí s jednou vrstvou typu LSTM o 4096 jednotkách, která zpracovává informace o aktuálním stavu hry pomocí API poskytnutého vývojáři Dota 2. Tento stav je zakódován jako vektor přibližně 20,000 číselných hodnot, které popisují prostředí z pohledu daného agenta.

Rozhodování probíhá prostřednictvím několika nezávislých akčních hlav, z nichž každá odpovídá za jiný aspekt chování — například volbu konkrétní akce, její umístění na prostorové mřížce nebo zpoždění provedení o určitý počet ticků. Tyto hlavy generují osm výstupních hodnot pro každé rozhodnutí, což umožňuje širokou škálu koordinovaných akcí bez nutnosti využívat lidská trénovací data.

Trénink probíhá na interní infrastruktuře Rapid, která umožňuje rozsáhlý paralelní výpočet a koordinaci tisíců strojů. Díky tomu OpenAI Five provádí trénink skrze masivní self-play. Do roku 2018 systém odehrává ekvivalent více než 180 let herního času, a to na 256 GPU a 128,000 CPU jádrech. Schéma tohoto výpočetního procesu ilustruje obrázek 1.6.



Obrázek 1.6: Architektura tréninkového systému OpenAI Five. Rollout workeři generují data ze zápasů, která jsou následně zpracována optimalizačním modulem využívajícím metodu PPO. Výsledné aktualizace jsou sdíleny napříč instancemi a evaluovány ve specifických herních prostředích.[25]

Díky své robustnosti, efektivitě a jednoduché integraci do distribuovaných systémů je PPO mimořádně vhodný pro tuto aplikaci. Umožňuje rozsáhlý paralelní trénink bez přeučení, přičemž jeho kompatibilita se sdílenými politikami, hodnotovými funkcemi a rekurentními vrstvami typu LSTM přispívá k jeho úspěchu v komplexním prostředí, jakým je Dota 2.

1.2.3 Voyager

Voyager[12] představuje agenta umělé inteligence, který propojuje výhody velkých jazykových modelů (LLM) se schopností samostatného učení ve složitých, otevřených prostředích. Na rozdíl od předchozích přístupů, které spoléhají na ručně vytvořené cíle, odměnové funkce nebo specificky upravená prostředí, Voyager operuje v původní verzi hry Minecraft — bez jakékoli lidské intervence nebo ručního ladění. Jedná se tak o příklad tzv. ztělesněného agenta (embodied agent), který nejen interpretuje herní svět, ale zároveň v něm jedná, sbírá zkušenosti a postupně se zlepšuje.

Volba hry Minecraft není náhodná. Jde o jedno z mála prostředí, která kombinují otevřenost, dlouhodobé cíle, nutnost plánování, nelineární obtížnost a procedurálně generované světy. Neexistuje zde žádný „konec hry“ a úspěch se nedá snadno kvantifikovat. To činí z Minecraftu ideální testovací prostředí pro dlouhodobé učení bez explicitní odměny. Agent musí samostatně odhalovat základní principy světa — od těžby a výroby nástrojů, přes přežití v noci, až po složité postupy jako farmaření nebo výroba lektvarů.

Aby toho dosáhl, Voyager využívá kombinaci tří klíčových mechanismů, které společně umožňují otevřené, samostatné a škálovatelné učení. Prvním z nich je automatická tvorba kurikula. Agent si sám generuje posloupnost úkolů pomocí velkého jazykového modelu GPT-4[2], přičemž tyto úkoly vždy vycházejí ze současného stavu světa, inventáře, vybavení, aktuální lokace a historie předchozích úspěchů i selhání. Místo toho, aby agent postupoval podle předem definovaného scénáře, sám si formuluje dosažitelné a smysluplné cíle — například „vytvoř kamenný krumpáč“, „postav pec“ nebo „zabij zombíka“.

Tyto úkoly nejsou náhodné. GPT-4 při jejich generování zohledňuje různé faktory, jako je biom, čas dne, přítomnost entit v okolí nebo například to, jaké úkoly již byly splněny. Díky tomu agent postupuje přirozeným tempem, které reflektuje jeho aktuální schopnosti a možnosti. Tento mechanismus vytváří dynamické, adaptivní kurikulum, které maximalizuje diverzitu dovedností bez nutnosti ručně nastavovat, co má být cílem.

Každý úkol, který agent úspěšně dokončí, se následně převádí do podoby samostatného programu napsaného v JavaScriptu. Tyto programy představují „dovednosti“ (skills), které Voyager ukládá do tzv. knihovny dovedností. Tato knihovna není statická — každá nově naučená dovednost se indexuje vektorově pomocí embeddingu popisu, který vytváří jazykový model GPT-3.5[2]. Tím vzniká databáze, ve které může agent v budoucnu vyhledávat relevantní programy, jež mu pomáhají řešit nové úkoly.

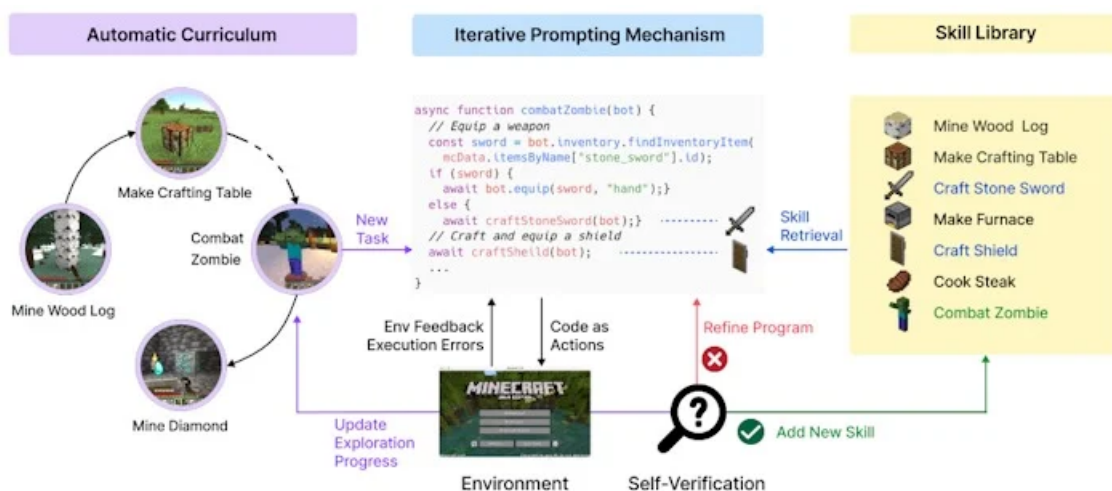
Dovednosti jsou konstruovány hierarchicky. Složitější akce (např. „vytvoř železný krumpáč“) využívají jednodušší dovednosti jako „vypal železnou rudu“, která sama závisí na dovednostech „těž železnou rudu“ a „vyrob pec“. Tento kompoziční přístup umožňuje agentovi stavět na tom, co se již naučil, a minimalizuje nutnost začínat znovu. Zároveň eliminuje problém tzv. katastrofického zapomínání, který je běžný u standardních metod zpětnovazebního učení.

V rámci samotného učení Voyager nevyužívá klasický přístup typu „pozorování → akce“, ale spoléhá na iterativní promptování. Při řešení úkolu agent nejprve navrhuje program, který daný úkol řeší. Tento program se následně provádí v prostředí a výstup — včetně chyb a výjimek — se znovu předkládá GPT-4. Na základě této zpětné vazby model upravuje prompt, opravuje program a zkouší jej znovu. Tento proces se opakuje, dokud tzv. sebe-verifikátor, opět založený na GPT-4, nevyhodnotí, že je úkol úspěšně splněn.

Iterativní přístup se ukazuje jako klíčový zejména v případech, kdy agent naráží na nekonzistence mezi plánem a realitou. Například pokud program selže s chybou typu „I cannot make a stick because I need: 2 more planks“, systém automaticky provádí korekci a aktualizuje plán. Pokud se problém opakuje (například při čtyřech po sobě jdoucích neúspěšných pokusech), agent daný úkol opouští a generuje si nový.

Ve srovnání s předchozími přístupy, jako jsou ReAct, Reflexion nebo AutoGPT, dosahuje Voyager výrazně lepších výsledků. V prostředí MineDojo získává $3,3\times$ více unikátních předmětů, prozkoumává svět do větší vzdálenosti ($2,3\times$ delší trasy) a dosahuje technologického pokroku až $15,3\times$ rychleji. Navíc jako jediný agent zvládá tzv. zero-shot úlohy, kdy je přenesen do nového světa s dosud neřešeným úkolem — například vytvoření dalekohledu nebo nalezení diamantů.

Díky kombinaci autonomního generování úkolů, kompoziční knihovny dovedností a schopnosti iterativní korekce chyb se Voyager přibližuje představě agenta, který je schopen dlouhodobého, otevřeného učení bez potřeby lidského zásahu. Schéma na obrázku 1.7 přehledně znázorňuje propojení těchto tří klíčových komponent. V prostředí, které není explicitně navrženo pro AI trénink, vykazuje chování, které je zároveň zvědavé, adaptivní i efektivní — a představuje tak významný krok směrem k obecné umělé inteligenci v otevřeném světě.



Obrázek 1.7: Vizualizace propojení hlavních komponent systému Voyager.[26]

Kapitola 2

Použité technologie a návrh hry

2.1 Použité technologie

2.1.1 Unreal Engine

Prostředí i agent se vytvoří a použije v Unreal Engine 5.2, který zajišťuje mapu i herní mechanismy s moderními renderovacími funkcemi v reálném čase. Některé z použitých prvků ve hře (hlavně modely a animace) pocházejí z FABu — nové online platformy od Epic Games, která spojuje starý Unreal Marketplace, Sketchfab, Quixel a ArtStation v jednotné řešení.

Unreal Engine (zkráceně UE)[17], vyvíjený společností Epic Games, vzniká z více než dvacetileté evoluce začínající v roce 1998 a dnes představuje špičkový engine pro tvorbu her, filmů i vizualizací. Jeho pružnost, nastavitelnost a výkon si získávají uznání vývojářů po celém světě.

Mezi výrazné přednosti UE patří systém Blueprint vizuálního skriptování — nadstavba nad C++, která umožňuje vytvářet herní logiku, interakce a AI pomocí uzlů a grafů místo klasického kódu v C++. Díky tomu je vývoj her přístupnější i kreativcům bez programátorského zázemí. Když jsou uzly přehledně poskládány, může být práce s nimi dokonce rychlejší pro prototypování a nasazení systémů než použití čistého C++.

Značnou oblibu nachází spíše UE5 než UE obecně — a to nejen mezi herními vývojáři, ale i ve filmové produkci, architektonické vizualizaci, produktovém designu nebo v průmyslu a tréninkových simulacích. Filmové týmy využívají UE5 pro virtuální natáčení v simulovaných prostředích, architekti jej používají v interaktivních prezentacích klientům a v medicíně či obraně slouží k tréninku zaměstnanců ve vysoce realistických scénářích.

Důvody volby Unreal Engine oproti alternativám jako Unity:

1. C++:

- C++ je jedním ze dvou hlavních jazyků v Unreal Engineu a umožňuje velmi rychlý kód, což je klíčové pro aplikace běžící v reálném čase, jako jsou hry. Navíc je v něm největší zkušenost vývojového týmu (v porovnání s C#, který je standardní v Unity).

2. Programovací nadstavba Blueprint:

- Blueprint je vizuální nadstavba nad C++, která výrazně zjednodušuje tvorbu herní logiky, interakcí a AI. Díky ní mohou tvořit i uživatelé s omezenou znalostí programování — někteří pracovníci s minimálními schopnostmi dosahují produktivity vyšší než při čistém C++.

3. FAB:

- Platforma FAB, kterou provozuje Epic Games, sjednocuje dřívější tržiště Unreal Marketplace, Sketchfab, Quixel a ArtStation. Díky tomu poskytuje široké spektrum assetů (modely, animace...), což výrazně urychluje vývoj bez nutnosti tvořit každou položku od nuly.

4. Pluginy:

- Unreal Engine umožňuje snadné rozšíření funkcionality prostřednictvím pluginů. Ty lze instalovat přímo z oficiálního obchodu, nebo je přidávat ručně do instalační složky UE — flexibilní řešení pro specifické funkce.

5. 3D architektura:

- UE je navržen od základu pro komplexní 3D tvorbu. V Unity nebo Godotu lze sice vyvíjet 3D aplikace, ale těchto nástrojů se častěji využívá pro 2D či 2.5D projekty kvůli nižší náročnosti.

Nevýhody Unreal Engineu oproti Unity či Godotu:

1. Menší komunita:

- Unity má širší uživatelskou základnu, což zajišťuje větší dostupnost řešení specifických problémů. Godot je v tomto ohledu srovnatelný s Unreal Engineem.

2. Vyšší hardwarové nároky:

- Unreal Engine 5 pro optimální výkon obvykle vyžaduje 32–64GB RAM, moderní vícejádrový procesor a grafickou kartu minimálně RTX4080, ideálně RTX4090. Epic Games standardně využívá pracovní stanice s 128–256GB RAM, procesory AMD Threadripper a grafikami RTX4080 pro efektivní zpracování rozsáhlých scén. Naproti tomu Unity běží pohodlně i na konfiguraci se 6jádrovým CPU, 16GB RAM a grafickou kartou GTX1060.



Obrázek 2.1: Ikony použitých technologií

2.1.2 Visual Studio

Visual Studio je používáno jako hlavní vývojové prostředí pro programování v C++ v rámci tohoto projektu. Unreal Engine 5 vyžaduje Visual Studio pro jakoukoli práci s C++, protože jde o úzce integrovaný nástroj — bez něj nelze kompilovat ani spouštět čistě C++ moduly.

Visual Studio je komplexní integrované vývojové prostředí (IDE) vytvořené společností Microsoft. Ačkoli podporuje více programovacích jazyků, při práci s Unreal Enginem se využívá zejména pro C++. Nabízí třeba správu projektů, inteligentní doplňování kódu (IntelliSense), ladění a analýzu paměti.

Při používání Unreal Engineu Visual Studio automaticky stahuje a konfiguruje projektové struktury a závislosti. Využívá build systémy jako MSBuild nebo CMake a úzce spolupracuje se soubory Unreal projektu — díky tomu se nové C++ třídy a funkce správně sestavují a propojují s editorem.

Visual Studio umožňuje ladění v reálném čase, profilování výkonu a sledování využití paměti — klíčové nástroje při vývoji rozsáhlých projektů v UE5.

Při vývoji na Windows je Visual Studio pro Unreal prakticky nezbytné. Umožňuje efektivní práci s C++, řízení kompilace a hladký vývojářský proces při programování na úrovni engineu.

2.1.3 NevarokML

NevarokML[18] představuje pokročilý plugin pro Unreal Engine 5, který umožňuje přímou integraci strojového učení — konkrétně zpětnovazebního učení — do herních nebo simulačních projektů. Plugin je založen na knihovně Stable-Baselines3 a zpřístupňuje oblíbené RL algoritmy jako PPO, A2C, DDPG, DQN, SAC a TD3, které agentům umožňují učit se optimálnímu chování metodou pokus–omyl.

NevarokML podporuje jak jednoagentní, tak víceagentní interakce a nabízí možnost paralelního tréninku napříč odlišnými prostředími. To významně zrychluje učení a pomáhá agentům lépe generalizovat v různých scénářích.

Architektura NevarokML funguje na principu klient–server: Unreal Engine zde působí jako serverová část, která simuluje prostředí a chování agentů, zatímco pythonový klient provádí strojové učení pomocí algoritmů ze Stable-Baselines3.

Komunikace mezi Unreal Enginem a Python klientem probíhá přes síť nebo sockety s využitím JSON protokolů, které přenášejí informace o stavu prostředí (pozorování, odměny, akce). Jakmile se model natrénuje, výsledné váhy se převedou do formátu ONNX a nasadí se jako objekt typu NNEModelData přímo do Unreal Enginu skrze jeho systém Neural Network Engine.

ONNX[19] (Open Neural Network Exchange) představuje otevřený formát a ekosystém navržený tak, aby umožnil snadný přenos již natrénovaných modelů hlubokého učení mezi různými vývojovými frameworky, platformami a typy hardwaru — bez potřeby složité konverze nebo zdlouhavých procesů.

Neural Network Engine (NNE)[20] je plugin určený pro provádění inferencí neuronových sítí v reálném čase přímo v prostředí Unreal Engine. Plugin využívá formát ONNX, který umožňuje spouštění modelů exportovaných z běžně používaných frameworků strojového učení, jako jsou PyTorch, TensorFlow nebo MXNet, ve formátu .onnx bez nutnosti dalšího převodu.

1. Vytvoření prostředí:

- Práce začíná vytvořením Blueprint třídy děděné z NevarokMLEnvironment, kde se definují základní události OnInit, OnStep a OnReset, které řídí logiku prostředí a průběh epizod.

2. Trenérský modul:

- Následuje vytvoření Blueprintu děděného z NevarokMLTrainer, kde se nastavují pozorovací a akční prostory (OnConstruct) a spouští se samotný trénink (OnStart).

3. Implementace v úrovni:

- Trenér i prostředí se umísťují do scény v Unreal Enginu, přičemž trenér odkazuje na prostředí a celý trénink začíná při spuštění úrovně.

4. Sledování učení:

- Pro vizualizaci průběhu učení se může použít TensorBoard, který umožňuje sledovat vývoj odměn, změny v politice agenta nebo ladění hyperparametrů.

Plugin je dostupný pro Unreal Engine ve verzi 5.2 a po jeho aktivaci je nutné spustit i backend klienta a ověřit připojení mezi Unreal Enginem a Pythonovým prostředím.

Hlavní vlastnosti a využití:

1. Široký výběr algoritmů:

- Podpora různých typů RL algoritmů umožňuje ladit chování agentů podle konkrétního použití (např. hodnotové nebo policy-gradient přístupy).

2. Podpora Blueprintů i C++:

- Plugin lze plně využívat jak přes vizuální skriptování, tak i přes nativní C++ rozhraní pro pokročilejší zásahy.

3. Cross-platform nasazení:

- I když trénink typicky probíhá na Windows, natrénované modely je možné nasadit na jakoukoli platformu podporovanou Unreal Enginem.

4. Vhodné pro výuku a výzkum:

- Plugin je volně dostupný pro nekomerční využití, což z něj činí ideální nástroj pro akademické projekty, simulace nebo vývoj prototypů.

2.2 Návrh hry

2.2.1 Návrh prostředí hry

Prostředí, ve kterém bude autonomní agent fungovat, lze realizovat různými způsoby – od jednoduchých po technicky náročnější. Mezi přístupnější varianty patří využití herních enginů, jako je Unity nebo Unreal Engine, zatímco pokročilejší řešení zahrnuje programování vizualizace od základu pomocí nízkoúrovňových API, jako jsou OpenGL nebo DirectX.

V tomto projektu bude vizualizace agenta realizována v Unreal Engine formou střílečky z pohledu první osoby. Prvním krokem bude vytvoření herní mapy, která umožní volný pohyb hráče i postavy řízené umělou inteligencí. Mapa bude inspirována městským náměstím po bitvě – její střed bude tvořit park s bankovní budovou. Park bude téměř prázdný kvůli zpusťování a bankovní budova bude obklopena bariádami. Právě v této budově se hráč po spuštění hry objeví. Bude zde možné nalézt zbraně a box se zbraněmi a brněním, které hráč využije v boji.

Kromě parku bude mapa zahrnovat i zastavěnou oblast – uzavřenější prostor obklopený budovami, který bude představovat vyšší riziko kvůli omezené viditelnosti a pohybu. Uvnitř zabarikádované zóny se budou nacházet výchozí body (spawn pointy) pro nepřátelské postavy ovládané AI.

Pro vyvážení obtížnosti budou nepřátelé po smrti odhazovat sbíratelné předměty, jako jsou lékárničky a zásobníky munice, které hráč využije k doplnění zdraví nebo střeliva.

Součástí uživatelského rozhraní bude tzv. HUD (Head-Up Display), který bude zobrazovat klíčové herní informace – aktuální stav zdraví, zásoby energie, počet nábojů a další. Z tohoto rozhraní bude možné přistupovat jak k hlavnímu menu, tak k pauzovací nabídce.

Celá hra bude strukturována do kolových vln. Po spuštění prvního kola se zavřou dveře banky, přičemž venku bude umístěna další bedna s vybavením, která umožní výměnu zbraní a brnění. V každém kole bude úkolem hráče eliminovat předem daný počet nepřátel. Po jejich porážce se aktuální kolo uzavře a automaticky začne nové – s vyšší obtížností nebo větším počtem protivníků. Tím vznikne dynamický cyklus, který otestuje schopnosti hráče i chování autonomního agenta v různých podmínkách.

Část prostředí a kódu u některých Blueprintů byla převzata z mého předchozího projektu. Vzhledem k rozsáhlým úpravám však bude v následující kapitole podrobně popsána struktura jednotlivých částí hry a následně samotného agenta.

2.2.2 Návrh agenta

Cílem je vytvořit agenta řízeného pomocí technik strojového učení, který se dokáže adaptovat v herním prostředí. Měl by být schopen se v prostředí zorientovat, přiblížit se ke svému cíli (hráči) a efektivně využít střelnou zbraň k jeho eliminaci. Tento agent nebude ovládán předem nadefinovanými skripty, ale bude se učit na základě zkušeností pomocí zpětnovazebního učení. Učící algoritmus bude založen na metodě Proximal Policy Optimization (PPO).

Celkový algoritmus agenta vytvořeného pomocí NevarokML je schematicky znázorněn na obrázku 2.2. Tento vývojový diagram ilustruje jednotlivé fáze interakce agenta s prostředím — od sběru pozorování, přes předání dat do neuronové sítě, predikci a provedení akce, až po získání odměny, uložení zkušeností a následný trénink modelu.



Obrázek 2.2: Vývojový diagram algoritmu agenta v rámci NevarokML

Vzhledem k tomu, že framework NevarokML neumožňuje přímou modifikaci architektury neuronové sítě, jsou všechna zlepšení zaměřena na okolní prvky smyčky — zejména strukturu pozorování, množinu dostupných akcí a definici odměn. Tyto prvky přímo ovlivňují kvalitu politiky, kterou agent získává.

Aby měl agent dostatečný přehled o situaci, budou mu prostřednictvím funkce `CollectObservations` předávány relevantní informace, na jejichž základě bude činit rozhodnutí. Tyto pozorování mu umožní vyhodnotit aktuální stav prostředí a adekvátně reagovat.

Agent bude mít přístup ke třem základním typům akcí:

- Pohyb,
- Rotace,
- Střelba – za předpokladu splnění podmínek (např. není aktivní cooldown, hráč se nachází v zorném poli apod.).

Tyto akce budou implementovány ve funkci `ApplyActions`, která interpretuje výstup neuronové sítě jak během tréninku, tak i v nasazení.

Jádrem efektivního učení bude systém odměn implementovaný ve funkci `HandleRewards`, jenž určuje, jak bude agent penalizován nebo odměňován za své chování.

Bude například odměňován za správnou rotaci, pohyb směrem k cíli nebo udržování optimální vzdálenosti. Zároveň bude trénink omezen tak, aby agent nebyl motivován pouze maximalizovat počet bodů bez reálného splnění cíle. Veškerá motivace bude přímo vázána na výsledek boje.

Na základě tohoto návrhu se očekává, že agent:

1. Rozpozná pozici cíle na základě poskytnutých pozorování,
2. Přiblíží se na efektivní vzdálenost,
3. Srovná rotaci tak, aby směřoval k cíli,
4. Zahájí palbu a pokusí se cíl eliminovat.

Tato posloupnost chování by měla emergentně vzniknout díky optimalizaci očekávané kumulativní odměny. Agent si rovněž bude moci po každém kole upravit své vybavení na základě konfigurace, kterou měl hráč v předchozím kole, aby zvýšil svou šanci na úspěch.

Kapitola 3

Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí

3.1 Základní logika hry

3.1.1 Hráčská postava

Po spuštění hry se aktivuje úvodní inicializační funkce, která spouští klíčové podsystémy hráčské postavy. Nejprve se připojuje mapa ovládacích vstupů (input map), která umožňuje hráči ovládat pohyb postavy prostřednictvím klávesnice. Následně se k postavě připojuje průhledový displej (HUD), jenž zobrazuje herní informace na obrazovce. Součástí inicializace je také aktivace zaměřovače, který se synchronizuje s HUDem a reaguje na vstupy hráče. Další funkce aktivují obnovovací systémy – konkrétně regeneraci zdraví, energie a kapacity baterie svítilny – a zpřístupňují odpovídající sekce HUDu.

Následující část inicializace vytváří proměnné potřebné pro propojení postavy v blueprintu s logikou sběru předmětů a obsluhy zbraní. Závěrečná fáze této funkce rozšiřuje HUD o prvek, který zobrazuje typ právě vybavené zbraně.

Pohyb kamery hráče se řídí pomocí pohybu myši – horizontálně po (lokální) ose Z a vertikálně po ose Y kamery. Samotný pohyb postavy se řídí dvěma samostatnými směry: vpřed/vzad a do stran. Oba směry využívají směrové vektory, které se následně předávají do funkce zajišťující fyzický posun postavy. Systém současně rozpoznává, zda hráč běží nebo se plíží, aby mohl spustit nebo zastavit odpovídající zvukové efekty.

Pro hladké ozvučení pohybu se využívá makro, které opakovaně volá funkci pro přehrávání zvuků kroků. Pokud hráč přestane padat (například po skoku), automaticky se znovu spouští zvuk chůze. Typ povrchu pod nohama se detekuje pomocí trasovací linky (ray trace) – podle zjištěného materiálu se přehrává odpovídající zvuk kroků.

Běhání hráčské postavy se váže na dostupnou energii (staminu). Pokud hráč není v podřepu ani nestojí na místě, ale začne se pohybovat a disponuje dostatkem staminy, systém automaticky aktivuje běh. Tím se spustí postupné snižování energie a současně se změní tempo přehrávání krokového zvuku. Jakmile běh ustane, energie se začne pozvolna obnovovat a rytmus kroků se upraví odpovídajícím způsobem.

Podobný princip platí pro úskok do stran. Pokud má hráč více než 15 bodů staminy a není v podřepu, může provést úskok ve směru aktuálně stisknuté pohybové klávesy. Po dokončení úskoku následuje krátká pauza, po které se energie začne regenerovat.

Při aktivaci plížení (podřepu) systém ověřuje, že hráč neběží, ani nevykonává úskok, a že má dostatek staminy. Pokud jsou tyto podmínky splněny, postava přechází do podřepu, zpomaluje tempo pohybu a začíná se doplňování energie. Při návratu do vzpřímené pozice se odečte část staminy, ale zároveň se znovu aktivuje její regenerace.

Součástí herní logiky je také systém pozastavení hry. Po stisknutí definovaného tlačítka se hra přeruší a na obrazovce se zobrazí pauzovací nabídka.

Mechanismus spotřeby a obnovy staminy funguje pomocí samostatných funkcí. Během běhu a dalších náročnějších akcí se každým herním tikem odečítá pevně stanovená hodnota staminy. Pokud hodnota klesne na nulu, systém automaticky deaktivuje běh. Pokud hráč zůstává neaktivní, spustí se režim obnovy, který energii doplňuje až do maximální hodnoty.

Hráčská postava disponuje také funkcemi pro správu zdraví. V případě zranění se okamžitě aktivuje odečet životů. Pokud postava stále žije, nastává fáze pomalé regenerace zdraví, která se zastavuje po dosažení maximální hodnoty. Pokud však zdraví klesne na nulu, postava umírá – veškeré ovládání se zablokuje, zvuky se deaktivují a zobrazí se obrazovka smrti s možností ukončení nebo restartu hry.

3.1.2 Zbraňový systém

Zbraňový systém ve hře simuluje chování střelných zbraní realistickým způsobem – včetně střelby, přebíjení, sběru munice a interakce s herním prostředím. Na začátku se inicializuje ukazatel na hráčskou postavu, což umožňuje přímé propojení mezi zbraní a hráčem.

Při výstřelu se automaticky odečte jeden náboj ze zásobníku a zároveň se spustí dvě animace – jedna vizualizuje samotný výstřel, druhá zobrazuje animaci hráče při střelbě. Pokud jsou náboje vyčerpány, zbraň vydává charakteristický zvuk „cvaknutí naprázdno“, který hráče upozorňuje na nutnost přebíjení.

Přebíjení probíhá pouze tehdy, pokud zásobník není plný. Systém nejprve zjišťuje, kolik nábojů chybí do plné kapacity, a následně je doplní z celkové zásoby. Celý proces doprovází přehrávání přebíjecí animace. Když hráč sebere na mapě náboje, systém ověří, zda nebyl překročen maximální limit – pokud ne, munice se přičte.

Interakce se zbraněmi probíhá pomocí dvou specifických tlačítek: klávesa E slouží ke sběru zbraní ze země, klávesa B umožňuje jejich odhození. Při sběru zbraně systém detekuje její přítomnost v okolí hráče, načte informace o typu zbraně, počtu nábojů v zásobníku i zásobě a tyto údaje přenesou do hráčova inventáře. Zbraň se poté odstraní ze scény a nahrazuje se aktivní zbraní, která je vizuálně připojena k hráči.

Při odhození zbraně se aktuálně držená zbraň zničí a před hráčem se vytvoří nová kopie, kterou lze znovu sebrat.

Logika střelby zohledňuje typ právě aktivní zbraně. Pokud má hráč automatickou pušku, hra umožňuje opakovanou střelbu po dobu držení spouště, dokud jsou k dispozici náboje. U brokovnice funguje náhodný rozptyl projektilů, který simuluje široký zásahový úhel typický pro tento typ zbraně – vystřelí se několik střel najednou, každá s mírně odlišnou trajektorií. V případě odstřelovací pušky se aktivuje klasická střelba jedním přesným výstřelem.

Střelbu spouští funkce, která vyhodnocuje rotaci a pozici hráčovy kamery. Ze zbraně se vysílá paprsek (raycast), který simuluje trajektorii střely a může zasáhnout nepřátelskou AI. Celý systém zohledňuje nepřesnosti způsobené pohybem hráče i náhodný rozptyl, zejména u brokovnice.

Po výstřelu se aktivuje simulace zpětného rázu, která upravuje směr pohledu hráče. Pokud střela zasáhne nepřítele, aplikuje se poškození. V případě zásahu objektu s fyzikálními vlastnostmi se na něj přenáší síla v opačném směru vůči směru výstřelu.

Každý výstřel spouští zvukový event, na který mohou nepřátelé reagovat. Zbraň zároveň generuje efekt vyhození prázdné nábojnice – ta vyletí ze zbraně a po určité době mizí ze scény.

Tlačítko R pro přebíjení volá funkci, která spouští proces doplnění zásobníku. Tím se zajišťuje plná integrace mezi hráčovými vstupy a systémem správy munice.

Celý zbraňový systém je úzce propojen s HUDem, který hráči poskytuje vizuální zpětnou vazbu o typu zbraně, aktuálním počtu nábojů a stavu zásob.

Agent využívá obdobný, avšak zjednodušený zbraňový systém – neobsahuje všechny funkcionality a některé modifikační efekty chybí.

3.1.3 Logika řízení herních kol

Logika řízení herních kol a nepřátelských jednotek začíná inicializací klíčových proměnných. Po spuštění úrovně systém nastavuje ukazatel na hráče, obnovuje hlasitost zvukových efektů podle posledního uloženého nastavení a volá funkci pro zahájení nové herní seance.

V rámci této úvodní funkce se resetuje proměnná indikující, zda již byli všichni nepřátelé naspawnováni. Zvyšuje se čítač aktuálního kola a spouští se časovač, který po předem definované době aktivuje začátek daného kola.

Zahájení kola zahrnuje nastavení počtu nepřátel, kteří se mají v daném kole objevit. Následně se spouští časovač, jenž s definovaným zpožděním volá funkci pro výběr náhodného místa spawnování. Na tomto místě se poté vytvářejí instance nepřátelských jednotek, implementovaných jako trenéři neuronové sítě v rámci tréninkové logiky.

Každý spawnovaný nepřítel se generuje podle pozice a orientace z předem určených spawnovacích bodů, čímž se zajišťuje variabilita a nepředvídatelnost jednotlivých kol.

Současně běží opakovaná kontrolní funkce, která sleduje počet aktivních nepřátel. Jakmile systém detekuje, že jsou všichni eliminováni, aktivuje ukončení aktuálního kola.

Závěrečná funkce každého kola vyhodnocuje vybavení hráče — typ zbraně a nastavené brnění (logika brnění je popsána v další sekci) — a poté spouští další kolo.

3.1.4 Systém sbírání předmětů

Logika sběru herních předmětů využívá detekci kolize mezi hráčovou postavou a specifickou kolizní sférou, která obklopuje sbíratelné objekty. Jakmile dojde ke kontaktu těchto dvou komponent, spustí se funkce, která umožňuje interakci a zvednutí daného objektu. Pokud hráč oblast opustí, interakce se přeruší a předmět zůstává nedotčen.

Při sběru zbraní kolizní sféra kromě detekce kontaktu uchovává také informace o konkrétní zbraní – například její typ a počet nábojů v zásobníku i celkové zásobě. Po stisknutí příslušné klávesy sféra tyto údaje předá blueprintu hráče, čímž dojde k přenosu zbraně do jeho rukou a následnému odstranění objektu ze scény.

Pro sběr munice se používá obdobná logika. Jakmile hráč vstoupí do kolizní oblasti s municí, systém ověřuje, zda má dostatek volné kapacity pro daný typ nábojů (automatická puška, brokovnice, odstřelovací puška). Pokud ano, náboje se přičítají do odpovídající zásoby.

V opačném případě zůstávají na místě a mohou být sebrány později, až hráč získá volné místo. Stejný mechanismus se uplatňuje i při sběru lékárníček, které doplní zdraví.

3.1.5 Krabice s vybavením

Pokud hráči dojde munice nebo si přeje změnit typ brnění, stačí přistoupit ke kterékoliv bíle označené krabici a stisknout klávesu E. Otevře se nové herní rozhraní, ve kterém lze pomocí šipek vybrat, zda chce používat brokovnici, automatickou pušku nebo odstřelovačku. Stejně tak lze zvolit typ brnění: lehké, střední nebo těžké.

Střední brnění představuje neutrální variantu. Lehké brnění zvyšuje rychlost pohybu, ale zároveň zvyšuje i zranitelnost. Těžké brnění naopak zvyšuje odolnost, ale snižuje pohyblivost.

Jakmile je hráč s výběrem spokojený, klikne na tlačítko Apply a vrací se zpět do boje.

3.1.6 Herní rozhraní (HUD)

V hlavním menu má hráč k dispozici několik možností. Tlačítko Play spouští samotnou hru. Volba Controls otvírá obrazovku s vizuálně zobrazeným schématem ovládacích kláves. Možnost Options zpřístupňuje nastavení hlasitosti, kvality audia a vizuálních parametrů pro optimalizaci výkonu. Změny lze potvrdit tlačítkem Apply nebo zrušit pomocí Back. Tlačítko Quit Game slouží k ukončení hry.

Během hraní může hráč kdykoli stisknutím klávesy P otevřít pauzovací menu. To obsahuje stejné volby jako hlavní menu, s tím rozdílem, že tlačítko Play zde slouží k návratu do hry a Main Menu přesměrovává zpět do úvodní nabídky. Pokud hráč během hry zemře, zobrazí se speciální nabídka se třemi možnostmi: Restart (nové spuštění hry), Quit to Main Menu (návrat do hlavního menu) a Quit (vypnutí hry).

Hlavní herní HUD je rozdělen do několika oblastí. V levém dolním rohu se nacházejí tři informační lišty: horní zobrazuje stav baterie svítilny na zbrani, prostřední ukazuje úroveň zdraví a spodní indikuje množství energie postavy. Vedle těchto lišt se zobrazuje číslo aktuálního kola. V pravém dolním rohu je uveden počet nábojů v zásobníku, celkový počet náhradních nábojů a informace o aktuálně držené zbrani (název a typ).

Střed obrazovky je vybaven zaměřovačem, jehož velikost se dynamicky mění v závislosti na rychlosti pohybu hráče, což přispívá k přesnějšímu míření. V horní části displeje se nachází časovač, který odpočítává čas do začátku nového kola. V pravém horním rohu je umístěn panel zobrazující změny atributů nepřátel generované neuronovou sítí, a nad ním se nachází počítadlo snímkové frekvence (FPS) pro sledování výkonnosti hry.

3.2 Autonomní agent

3.2.1 První verze agenta

Pro začátek testuji samotný proces tréninku, tvorby modelu a následného nasazení natrénovaného modelu do simulace pomocí pluginu NevarokML. V první verzi se rozhoduji vytvořit jednoduchého agenta, který se dokáže adaptivně pohybovat a v průběhu tréninku i simulace postupně zlepšuje svou schopnost dosáhnout stanoveného cíle.

Trénovací mapa

Na trénovací mapě se nachází hlavní entita typu NevarokMLTrainer, která slouží k tréninku agenta v rámci pluginu NevarokML. Pod ní funguje 270 podřízených entit typu NevarokMLEnvironment, z nichž každá obsahuje simulovaný herní svět pro konkrétního agenta. V každém prostředí je zároveň přítomna entita agent, na níž probíhá samotný trénink nebo simulace.

Každé prostředí obsahuje pouze podlahu (bez stěn – v této první verzi nejde o uzavřený prostor), pod kterou se nachází spouštěcí oblast typu Trigger s názvem AbyssTrigger. Kromě toho je poblíž každého agenta umístěn ještě jeden Trigger s názvem GoalTrigger, který slouží jako cíl simulace.

Nevarok trenér

V NevarokMLTraineru se nejprve nastavuje velikost tzv. Action Space (akčního prostoru), aby NevarokMLEnvironment věděl, kolik typů akcí může agent provádět – buď jednu po druhé, nebo, pokud je to nastaveno, i několik akcí během jednoho tiky tréninku či simulace. V první verzi, kdy agent zvládá pouze pohyb, obsahuje akční prostor pět akcí, které provádí sekvenčně.

Následně se připravuje tzv. Observation Space (pozorovací prostor), do kterého agent během hry ukládá určená data sloužící k učení a rozhodování v průběhu tréninku a simulace. Po vytvoření Action Space a Observation Space se aktivuje Blueprint uzel nazvaný PPO, což představuje algoritmus zvolený pro všechny tři verze tréninku a simulace. Tento uzel definuje klíčové hyperparametry:

- „Policy = MlpPolicy“ — agent používá plně propojenou vícervrstvou perceptronovou síť pro aproximaci politiky i hodnotové funkce.
- „Learning Rate“ — ovlivňuje velikost kroků při aktualizaci vah, nižší hodnota znamená stabilnější, ale pomalejší učení.
- „N Steps“ — počet kroků simulace sbíraných před aktualizací politiky.
- „Batch Size“ — počet přechodů (stav–akce–odměna) zpracovaných v jednom mini-batchi.

- „N Epochs“ — kolikrát se síť trénuje na nasbíraných datech před jejich zaházením.
- „Gamma“ — diskontní faktor pro budoucí odměny.
- „GAE Lambda“ — vyvažuje bias a variance při výpočtu výhodnosti pomocí Generalized Advantage Estimation.
- „Clip Range“ — omezuje změny politiky při aktualizaci, aby se zabránilo nestabilnímu učení.
- „Ent Coef“ — zvyšuje exploraci odměňováním vyšší entropie politiky.
- „Vf Coef“ — určuje váhu hodnotové funkce v celkové ztrátě.
- „Max Grad Norm“ — slouží k ořezání gradientů a prevenci divergence.

Volitelné parametry zahrnují: „Use SDE“ pro aktivaci stavového šumu, „Sde Sample Freq“ pro určení jeho frekvence, a „Verbose“ pro zapnutí výpisu logů během tréninku.

Po uzlu PPO následuje uzel Learn, který spouští samotný trénink nebo simulaci pomocí zvoleného algoritmu:

- „Device“ — umožňuje volbu mezi CPU a GPU.
- „Timesteps“ — celkový počet tréninkových kroků.
- „Eval Eps“ — počet evaluačních epizod.
- „Save Freq“ a „Log Interval“ — určují frekvenci ukládání modelů a zapisování logů.
- „Load Model Path“ a „Save Model Name“ — správa checkpointů a ukládání nových verzí.
- „Deterministic“ — zajišťuje deterministické chování při vyhodnocení po tréninku.

Volitelné ladicí volby jako „Show Tensorboard“, „Show Reward“, „Show Step Debug“ a „Show Reset Debug“ umožňují vizualizaci a průběžné sledování učení i chování agenta.

Nevarok prostředí

Po inicializaci prostředí se nejprve spouští událost „Event On Init“. Ta kontroluje, zda je aktivní proměnná „Simulate“, která určuje, zda prostředí začíná od začátku, nebo pokračuje s uloženým modelem. Pokud je podmínka splněna, vytváří se instance uzlu „NNEModule“, který načítá a inicializuje uložený model. Tento uzel vyžaduje vstupy z „Action Sample“ a „Observation Sample“, aby model věděl, s jakými akčními a pozorovacími daty bude pracovat.

Výstup modelu se následně ukládá do proměnné „Inference Model“, která slouží pro predikci akcí agenta. V této fázi se rovněž ukládá počáteční pozice agenta do proměnné „Agent Initial Location“, získaná z uzlu „Get Actor Location“. Tato pozice se používá pro resetování prostředí na začátku každé epizody. Agent se zároveň přidává do seznamu „Ignore Actors“ pomocí uzlu „Add Unique“, aby byl ignorován při raycast kolizích během simulace.

Po inicializaci následuje průběžná simulace skrze událost „Event On Step“. Pokud je „Simulate“ stále aktivní, spouští se uzel „Predict“, který navrhuje akce pro aktuální stav prostředí. Tyto akce se aplikují pomocí uzlu „Apply Actions“, zatímco aktuální stav se zaznamenává přes uzel „Collect Observations“. Odměny za jednotlivé akce vyhodnocuje funkce „Handle Rewards“ a volitelně se zobrazuje ladicí výstup prostřednictvím uzlu „Debug“, který umožňuje sledování simulace v reálném čase.

Mezi hlavními ticky definovanými parametrem „Trainer->TickInterval“ se může spustit událost „Event On Step Skip“. Ta slouží například k aplikaci akcí v jemnějších krocích nebo k vyplnění mezer mezi hlavními kroky. Pozorování získaná během „Step Skip“ se však do tréninkového procesu nezahrnují.

Nakonec se spouští událost „Event On Reset“, která se volá před prvním krokem epizody a po jejím dokončení. Slouží k resetování prostředí do výchozího stavu. Nejprve se aktivuje ladicí uzel „Debug“, který slouží k indikaci poslední získané odměny. Následně uzly „Reset Triggers“ a „Reset Agent“ obnovují stav agentů i spouštěcích oblastí. Uzel „Clear“ vymaže nasbírané akce a pozorování („Action Sample“), aby se nová epizoda mohla začít sbírat od nuly. Na závěr funkce „Collect Observations“ ukládá počáteční stav prostředí, jenž slouží jako výchozí bod pro trénink nebo simulaci.

Tento hlavní řetězec událostí zajišťuje kompletní cyklus tréninku a simulace agenta — od inicializace přes jednotlivé kroky s predikcemi modelu až po resetování prostředí na začátku nové epizody.

Při spuštění sběru pozorování („Collect Observations“) z hlavního cyklu se nejprve nastavuje proměnná „Index Counter“ na výchozí hodnotu (0, 0), která reprezentuje indexy pro iteraci senzorů v mřížce. Tato proměnná se pak postupně inkrementuje pomocí funkčních bloků „Index Plus Plus“, přičemž každé volání posouvá index o jeden krok ve směru osy Y (parametr „Step dx = 0, dy = 1“). Tento mechanismus zajišťuje sekvenční průchod jednotlivými senzory nebo směry detekce.

Během každé iterace se načítá aktuální pozice agenta a cílového senzoru. Pomocí uzlu „Get World Location“ se získává pozice senzoru, a pomocí „Get Actor Location“ se načítá aktuální poloha agenta. Funkce „Find Look at Rotation“ vypočítává rotaci směrem k cílovému bodu, která se následně aplikuje na senzor pomocí uzlu „Set World Rotation“. Tento postup zajišťuje, že senzor vždy směřuje správným směrem před samotným trasováním.

Následně probíhá vzorkování pomocí uzlu „Sample Box Line Trace“, který funguje jako raycast s definovanou délkou a nastaveným kanálem („Trace Channel = Visibility“). Parametry „Difference Multiplier“ a „Velocity Multiplier“ škálují rozdíly vzdáleností a rychlosti, které se ukládají do pozorovacího prostoru. K uzlu jsou rovněž připojeny seznamy „Ignore Actors“ a volitelně „Trace Tags“, jež definují objekty ignorované při kolizi a typy vyhodnocovaných zásahů.

Výstupy z raycastu se zapisují do „Observation Sample“, přičemž obsahují indexy jako „Velocity Length Index“, „Difference Length Index“, „X/Y/Z Velocity Index“, „Direction Index“ a „Normal Index“. Tyto indexy reprezentují složky rychlosti, směrové vektory a normály zásahu, které se následně používají jako vstupní data pro neuronovou síť během tréninku nebo inferencí. Výstupy, které se dále nezpracovávají, směřují na pin „Unused“.

Pro účely vizualizace a ladění slouží volitelné parametry na pravé straně uzlu: „Debug Trace“ umožňuje zobrazení paprsků přímo ve scéně, „Debug Duration“ určuje délku jejich zobrazení, „Debug Color“ nastavuje barvu a „Debug Log“ zajišťuje textový výstup do konzole.

Celý tento mechanismus umožňuje agentovi sbírat strukturovaná pozorování z okolního prostředí prostřednictvím senzorů a line trace. Tato data se následně předávají do pozorovacího prostoru, kde slouží k rozhodování a učení.

Po obdržení pokynu z hlavního řetězce se spouští událost „Apply Actions“, která převádí diskrétní akce agenta na pohyb v herním světě. Nejprve se prostřednictvím uzlu „Enable Movement Tick“ aktivuje pohybový tick komponenty „Nevarok ML Character Input“. Tento krok zajišťuje, že se v aktuálním kroku správně zpracovávají pohybové vstupy.

Následně se pomocí uzlu „Get Discrete Value“ získává hodnota diskrétní akce ze struktury „Action Sample“. Tato hodnota určuje, jakou konkrétní akci agent provede. Výsledná hodnota se následně předává do uzlu „Add Discrete World Move Input“, který ji mapuje na odpovídající směr pohybu v rámci herního prostředí:

- „Left (1)“ — pohyb vlevo,
- „Forward Left (-1)“ — diagonálně vlevo vpřed,
- „Forward (2)“ — vpřed,
- „Forward Right (-1)“ — diagonálně vpravo vpřed,
- „Right (3)“ — vpravo,
- „Backward Right (-1)“ — diagonálně vpravo vzad,

- „Backward (4)“ — vzad,
- „Backward Left (-1)“ — diagonálně vlevo vzad.

V tomto případě je agentovi povolen pouze pohyb vpřed, vzad nebo do stran — diagonální pohyb není možný.

Volba „Grounded Only“ omezuje aplikaci pohybu pouze na situace, kdy se agent nachází v kontaktu se zemí. Po zpracování akce se pomocí uzlu „Force Movement Tick“ vynucuje provedení pohybového snímku, aby se změna polohy okamžitě promítla do simulace.

Tento mechanismus umožňuje převádět diskrétní rozhodnutí neuronové sítě na konkrétní pohyb agenta v prostředí a zajišťuje, že každá akce má přímý vliv v rámci jednoho kroku simulace nebo tréninku.

Po obdržení pokynu z hlavního řetězce se spouští událost „Handle Rewards“, která vyhodnocuje odměny, penalizace a podmínky pro ukončení epizody. Logika této události je rozdělena do několika sekvenčně zpracovávaných částí.

První část představuje časový postih za každý krok („Penalty "Time/Step"“). Pomocí uzlu „Add Reward“ systém agentovi uděluje malou zápornou hodnotu, obvykle $-0,01$, čímž jej motivuje k dosažení cíle co nejrychleji a k minimalizaci zdržení během epizody.

Následuje kontrola podmínek pro ukončení epizody. První z nich představuje dosažení maximálního počtu kroků („Done condition "Time out"“). Systém porovnává počet aktuálních kroků („Steps“) s hodnotou „Max Steps“. Pokud je tato hodnota větší nebo rovna maximálnímu limitu, proměnná „Done“ se nastavuje na hodnotu `true` a epizoda končí kvůli vypršení časového limitu.

Další podmínku představuje pád agenta mimo herní prostor („Done condition "Fell out"“). Detekce probíhá pomocí uzlu „Is Triggered“, který je připojen k „Abyss Trigger“. V případě pádu epizoda okamžitě končí a agent obdrží penalizaci „Penalty "Fell out"“ = $-1,0$, čímž je motivován, aby se vyhýbal situacím vedoucím k pádu.

Poslední vyhodnocovanou podmínkou je dosažení cíle („Done condition "Target reached"“). Uzel „Is Triggered“, připojený k „Goal Trigger“, detekuje vstup agenta do cílové oblasti. Pokud je cíl dosažen, proměnná „Done“ se aktivuje a agent získává kladnou odměnu „Bonus "Target reached"“ = $+1,0$, která podporuje opakování úspěšného chování.

Celá sekvence kombinuje:

- drobnou penalizaci za zbytečné zdržování,
- výraznou penalizaci za neúspěch,
- vysokou odměnu za splnění cíle,

čímž vzniká základní reward shaping, který pomáhá neuronové síti efektivněji učit se správnému chování v herním prostředí.

Součástí logiky prostředí jsou také pomocné funkce, které zajišťují správné resetování a ladění epizod.

Funkce „Reset Triggers“ obnovuje pozici cílového i propastného spouštěče na začátku každé epizody. Nejprve se generuje náhodný vektor v rovině XY pomocí uzlu „XY Random Vector“, který se normalizuje uzlem „Normalize“. Tento směr se násobí škálovacím faktorem (např. 500) a přičítá k aktuální pozici získané přes „Get Actor Location“. Výsledná pozice je nastavena pomocí uzlu „Set Actor Location“. Nakonec se volají uzly „Reset Trigger“ pro oba spouštěče („Goal Trigger“ i „Abyss Trigger“), čímž se zajistí jejich výchozí nastavení pro následující epizodu.

Funkce „Reset Agent“ obnovuje agenta do pozice uložené v proměnné „Agent Initial Location“. Touto pozicí se nastavuje agent pomocí „Set Actor Location“, a následně se uzlem „Reset Input“ vymažou všechna dosavadní přesměrování komponenty „Nevarok ML Character Input“. Tím je zajištěno, že agent vždy začíná každou epizodu z čistého výchozího stavu, bez zbytkového pohybu či vstupů.

Funkce „Debug“ slouží k vizuálnímu i textovému ladění průběhu epizody. Využívá uzel „Draw Debug String“, který zobrazuje nad agentem text s aktuální kumulovanou odměnou („Episode Reward“). Barva textu a délka jeho zobrazení („Duration“) jsou nastavitelné, což umožňuje snadné a rychlé sledování chování agenta během simulace nebo tréninku.

Tyto tři funkce – „Reset Triggers“, „Reset Agent“ a „Debug“ – společně zajišťují spolehlivé resetování prostředí, vynulování stavu agenta a intuitivní vizualizaci průběhu epizody.

Agent, GoalTrigger a AbyssTrigger

Tyto Actors neobsahují žádný vlastní kód — slouží výlučně k zajištění funkčnosti systému NevarokMLEnvironment.

Agent má několik senzorů orientovaných do stran, které sbírají informace o okolním prostředí. Dále obsahuje komponentu NevarokMLCharacterInput, umožňující jeho ovládání z prostředí NevarokMLEnvironment, a dva typy senzorů: standardní NevarokML sensor a sensor pro detekci zásahů.

Actor AbyssTrigger obsahuje obyčejný NevarokML sensor a trigger typu NevarokMLBoxTrigger, který při kontaktu s agentem ukončuje probíhající učící epizodu — jak je popsáno výše. Goal Trigger funguje obdobným způsobem, liší se však svou specifickou funkcí a využitím v herním prostředí.

Model první verze

Tabulka 3.1: Výsledky základního tréninku první verze agenta

Metrika	Hodnota
rollout/	
ep_len_mean	6.34
ep_rew_mean	0.937
time/	
fps	2502
iterations	20
time_elapsed	457
total_timesteps	1080000
train/	
approx_kl	0.0009790711
clip_fraction	0.01
clip_range	0.2
entropy_loss	-0.157
explained_variance	0.926
learning_rate	0.0003
loss	-0.00125
n_updates	3800
policy_gradient_loss	-0.00192
value_loss	0.00187

Výsledky základního tréninku první verze modelu zachycené v tabulce 3.1 ukazují statistiky dosažené po celkovém počtu 1080000 simulovaných kroků (**total_timesteps**) během 20 iterací. Průměrná délka epizody (**ep_len_mean**) činí 6.34 kroků a průměrná kumulovaná odměna na epizodu (**ep_rew_mean**) dosahuje hodnoty 0.937, což naznačuje, že agent se naučil úkol plnit s relativně vysokou úspěšností.

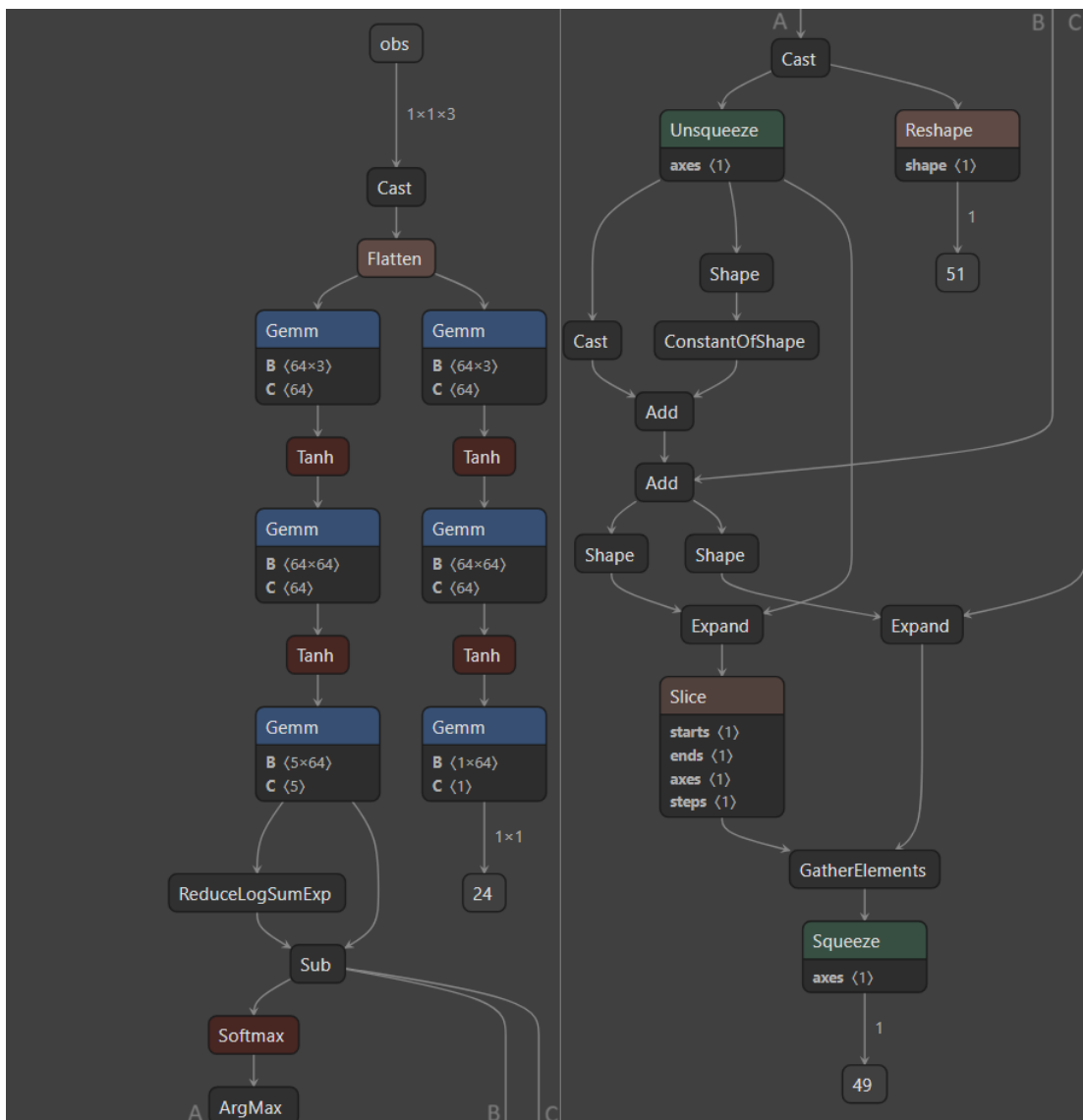
Trénink probíhá rychlostí přibližně 2500 snímků za sekundu **fps** a jeho celková doba trvá 719 sekund. Během tohoto období dochází k 3800 aktualizacím modelu **n_updates** při konstantní hodnotě **learning_rate** = 0.0003.

Hodnota přibližné KL divergence **approx_kl** je velmi nízká (0.00098), což naznačuje minimální odchylku nových politik od předchozích. Nízká hodnota **clip_fraction** = 0.01 ve spojení s **clip_range** = 0.2 potvrzuje, že zásahy do gradientu jsou vzácné.

Ztrátové funkce vykazují očekávané hodnoty: **policy_gradient_loss** = -0.00192, **value_loss** = 0.00187 a celková ztráta modelu **loss** = -0.00125. Hodnota **entropy_loss** = -0.157 odpovídá mírné míře exploração, která je během tréninku mírně penalizována.

Významným ukazatelem je také **explained_variance** = 0.926, což znamená, že přibližně 92,6% variability cílových hodnot je vysvětleno predikcemi hodnotové funkce — potvrzuje to dobrý odhad stavu a stabilitu tréninku.

Model funguje téměř stoprocentně (vzhledem ke své jednoduchosti, zaměřené pouze na pohyb agenta směrem k cíli). Stejně přesně se chová i při nasazení na herní mapu, kde nahrazuje standardní pohyb agenta tímto algoritmem.



Obrázek 3.1: Vzhled modelu z první verze agenta

Pro představu je na obrázku 3.1 zobrazen exportovaný ONNX model první verze. Model reprezentuje jednoduchou neuronovou síť typu MLP se dvěma skrytými vrstvami a převážně actor-critic architekturou. Vstupní pole „obs“ má tvar $1 \times 1 \times 3$, který se následně převádí uzlem „Cast“ a zplošťuje (Flatten) na trojici hodnot představujících stavové pozorování.

Síť se dále větví do dvou částí:

1. Policy head (levá větev):

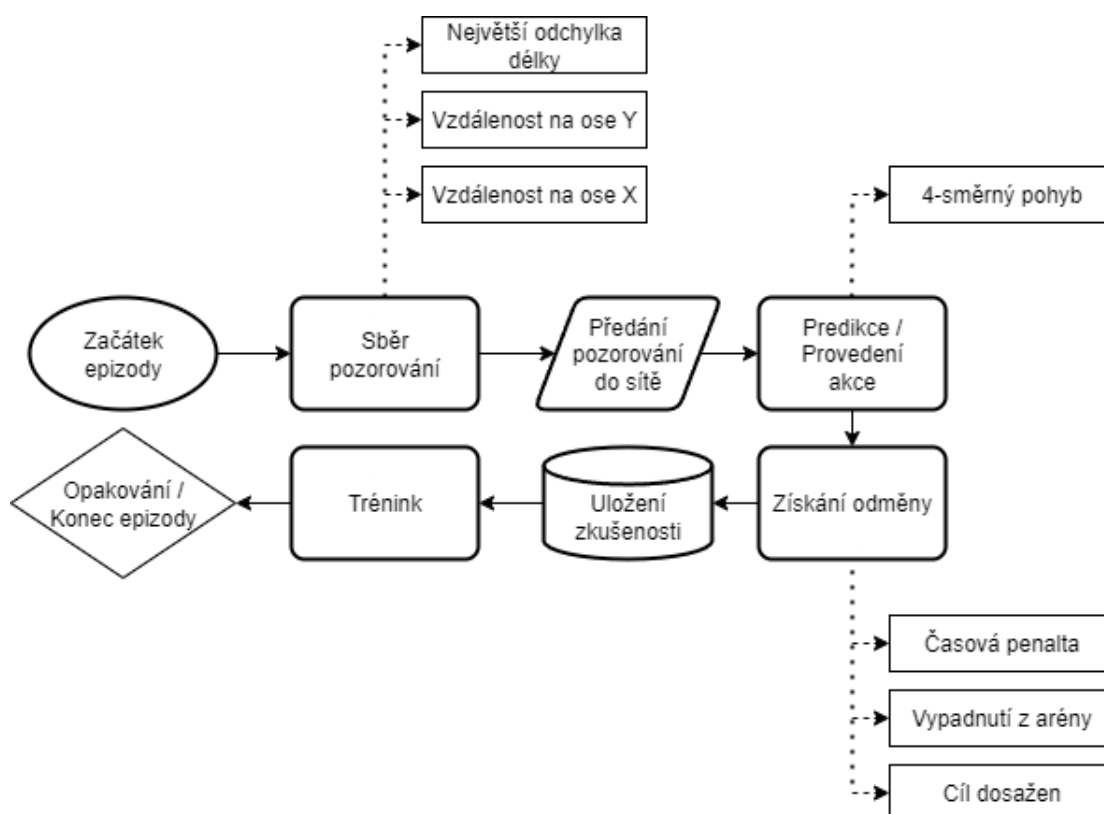
- obsahuje dvě plně propojené vrstvy („Gemm“ $3 \rightarrow 64$ a $64 \rightarrow 64$) s aktivací funkcí „Tanh“, následované výstupní vrstvou ($64 \rightarrow 5$) pro pět diskretních akcí. Výstup se dále zpracovává pomocí „ReduceLogSumExp“, normalizuje funkcí „Softmax“ a převádí pomocí „ArgMax“, který vrací index akce s nejvyšší pravděpodobností.

2. Value head (pravá větev):

- má stejnou strukturu skrytých vrstev ($3 \rightarrow 64$ a $64 \rightarrow 64$ s „Tanh“) a končí výstupní vrstvou ($64 \rightarrow 1$), která poskytuje skalární odhad hodnoty stavu (baseline pro algoritmus PPO).

V pravé části grafu se nacházejí také podpůrné uzly „Unsqueeze“, „Shape“, „Slice“, „GatherElements“, které slouží k úpravě tvarů tenzorů a indexování výstupů při inferenci napříč více instancemi prostředí.

Na obrázku 3.2 je uveden vývojový diagram algoritmu agenta, který shrnuje veškerou logiku implementovanou v této verzi.



Obrázek 3.2: Vývojový diagram algoritmu agenta – první verze

3.2.2 Druhá verze agenta

Druhá verze agenta již obsahuje kompletní logiku chování – dokáže se pohybovat, otáčet a nakonec i střílí na nepřítele.

Změny v trénovací mapě

Trénovací mapa se částečně mění. Místo podlahy bez stěn má nyní každé prostředí podlahu se stěnami, aby agent během základního tréninku nemůže spadnout mimo vymezenou oblast. Tato oblast je zároveň dvojnásobně větší. Agent na mapě zůstává beze změn, ale obě původní spouštěcí oblasti `GoalTrigger` a `AbyssTrigger` jsou odstraněny. Místo nich se na mapě nově nachází Actor s názvem `Enemy` (nepřítel).

Změny v trenérovi

V `NevarokMLTrainer` dochází k drobným úpravám. Trénér nyní nenastavuje velikost „Action Space“ pomocí jediné hodnoty, ale přijímá pole o třech prvcích: první s rozsahem 0–8, druhý 0–2 a třetí 0–1. `NevarokML` tak v pozadí vytváří akční krychli, která umožňuje agentovi provádět až tři akce současně namísto sekvenčního provádění. Celkový počet možných kombinací akcí se tím zvyšuje z 5 na 54.

Rozšiřuje se také „Observation Space“ — agent nyní sbírá o dvě další hodnoty navíc, což je nezbytné pro nové typy akcí.

Kromě toho se přidává ukládání počáteční rotace agenta, které se využívá při resetu a pro správnou orientaci v každé nové epizodě.

Změny v prostředí

Základní inicializační část `NevarokMLEnvironment` zůstává beze změny, ale ostatní části doznávají úprav.

Ve funkci „CollectObservations“ se přidává logika, která vypočítává vzdálenost a úhlovou odchylku (yaw rozdíl) mezi agentem a nepřítelem. Tyto hodnoty se následně zapisují do pozorovacího prostoru.

Nejprve se pomocí uzlu „Sample Box Line Trace“ detekuje vzdálenost k objektu před agentem. Výstup „Difference Length“ se ukládá do proměnné „Distance To Enemy“ a následně pomocí uzlu „Set Box Value“ do pozorovacího pole.

Současně probíhá výpočet směrového rozdílu mezi agentem a cílem. Uzel „Get Actor Rotation“ spolu s „Break Rotator“ extrahují hodnotu yaw agenta, zatímco funkce „Find Look at Rotation“ určuje požadovaný yaw směr k cíli. Tyto hodnoty se ukládají jako „Agent Yaw“ a „Look at Yaw“.

Úhlový rozdíl se dále normalizuje pomocí uzlu „Normalize Axis“, který zajišťuje, že výsledek zůstává v rozsahu $[-180^\circ, 180^\circ]$. Hodnota odchylky se následně ukládá jako „Yaw Diff“, opět pomocí „Set Box Value“.

Pro každý zápis do pozorovacího pole se využívá inkrementační čítač „Index Counter“, který pomocí funkce „Index Plus Plus“ zajišťuje, že jednotlivé hodnoty se zapisují do různých indexů bez přepisování.

Tato sekvence poskytuje agentovi nové prostorové informace důležité pro navigaci – konkrétně vzdálenost k cíli (nepříteli) a velikost směrové odchylky, kterou musí agent korigovat, aby se správně natočil ke svému cíli.

Funkce „Apply Actions“ nyní zpracovává výstup politiky agenta rozdělený do tří větví podle „Multi Discrete“ reprezentace: pohyb, rotace a střelba.

Pomocí uzlu „Sequence“ se zajišťuje sekvenční provedení všech tří akčních větví:

1. Pohyb:

- První větev zpracovává výstup s indexem 0 pomocí uzlu „Get Multi Discrete Value“ a výsledek předává do uzlu „Add Discrete World Move Input“. Tento uzel interpretuje hodnoty 0–8 jako pohyb vpřed, vzad, do stran a nově také diagonálně.

2. Rotace:

- Druhá větev (index 1) určuje otáčení — hodnoty 1 a 2 reprezentují rotaci vlevo či vpravo, přičemž realizaci zajišťuje uzel „Add Discrete Turn Input“.

3. Střelba:

- Třetí větev (index 2) aktivuje střelbu pouze při splnění dvou podmínek: agent se nachází dostatečně blízko cíli a zároveň je správně natočen. Pokud jsou obě podmínky splněny, vyvolá se uzel „Trigger Fire Action“, který realizuje samotnou logiku střelby.

Každou akci zprostředkovává komponenta „Nevarok ML Character Input“. Nejprve se aktivuje pohybový tick pomocí „Enable Movement Tick“ a po dokončení celé sekvence následuje „Force Movement Tick“, který zajišťuje okamžité aplikování akcí během jednoho simulačního kroku.

Funkce `Trigger Fire Action` představuje zjednodušený systém střelby, který se spouští po uplynutí 0,5s díky uzlu `Do Once` a opakovanému obnovení proměnné `Can Fire` přes `Set Timer by Function Name`. Kód nejprve ověřuje, zda je střelba povolena (`Can Fire == true`); pokud ano, pokračuje dále.

Ze scény se získávají pozice a směr kamery pomocí uzlů `Get World Location` a `Get World Rotation`, přičemž vektor směru se prodlužuje o určitou vzdálenost pomocí operace `+`. Tento výsledný směr se následně použije ve funkci `Line Trace by Channel`, která simuluje paprsek vpřed ve směru pohledu agenta a ignoruje samotného agenta.

Výsledek raycastu se zpracovává pomocí **Break Hit Result**. Pokud dojde ke kolizi (**Blocking Hit == true**) a objekt odpovídá nepříteli (**Hit Actor == Enemy**), agent získává odměnu +2,0 a spouští se funkce **Apply Damage** s hodnotou 10. Pokud je zásah mimo nepřítele nebo k žádnému zásahu nedojde, agent obdrží penalizaci -0,2.

Na závěr se proměnná **Can Fire** nastavuje zpět na **false** a opět se spouští časovač s hodnotou 0,5,s. Po jeho vypršení se střelba opět povoluje.

V novém systému odměn je agent průběžně motivován k dosažení cíle, správné orientaci, udržování optimální vzdálenosti od cíle a eliminaci nepřítele, přičemž je penalizován za chyby, neaktivitu nebo vypršení časového limitu.

1. Odměna za správné natočení:

- Pokud je rozdíl směru („YawDiff“) mezi agentem a směrem k nepříteli malý, agent získává odměnu až do výše 0,03 — čím přesněji míří, tím vyšší odměna. Při větším odchýlení ztrácí až -0,05.

2. Odměna za rotaci během pohybu:

- Agent je odměňován pouze tehdy, pokud se pohybuje vpřed a zároveň se otáčí směrem k cíli, což se detekuje pomocí skalárního součinu forward vectoru a směru k cíli. Odměna se škáluje až do výše 0,03.

3. Odměna za ideální vzdálenost:

- Agent získává mírnou odměnu (maximálně 0,03) za udržování optimální vzdálenosti od cíle. Pokud je příliš blízko nebo příliš daleko, je penalizován hodnotou -0,03.

4. Eliminace nepřítele:

- Pokud agent úspěšně eliminuje nepřítele, epizoda se ukončuje a je mu přidělena hlavní odměna ve výši 5,0.

5. Penalizace za timeout:

- Pokud agent epizodu nevyřeší a dosáhne maximálního počtu kroků, obdrží výraznou penalizaci -10,0, která slouží jako jasný signál selhání.

Funkce „Reset Triggers“ je nahrazena funkcí „Reset Enemy“, která funguje obdobně, avšak navíc nastavuje poškozenému nepříteli proměnnou „Death“ na **false** a obnovuje jeho zdraví na hodnotu 50.

Funkce „Reset Agent“ se kromě korekce počáteční rotace agenta nemění.

Agent samotný zůstává beze změn, nicméně je vytvořen nový actor „Enemy“, jenž obsahuje základní „NevarokML“ senzor a také kód pro zpracování poškození a případného úmrtí. Při zásahu se spouští událost `Event AnyDamage`, která volá vlastní událost `Health Modifier` s předanou hodnotou poškození. Tato událost nejprve odečítá hodnotu `Damage` od aktuální proměnné `Health` a poté výsledek ořezává pomocí funkce `Clamp (Float)` na rozsah `[0,0;Max Health]`, aby nedošlo k přetečení nebo záporné hodnotě. Upravená hodnota se ukládá zpět do proměnné `Health`.

Následuje kontrola, zda je zdraví menší nebo rovno nule. Pokud ano, aktivuje se větev `True`, která spouští uzel `Do Once`, a následně vlastní událost `Dead`. Tím se proměnná `Death` nastavuje na `true`, což signalizuje zabití nepřítele. Při vyhodnocení odměn to automaticky spouští reset prostředí.

Model druhé verze

Tabulka 3.2: Výsledky základního tréninku druhé verze agenta

Metrika	Hodnota
rollout/	
ep_len_mean	36.4
ep_rew_mean	13.8
time/	
fps	2502
iterations	59
time_elapsed	1273
total_timesteps	3240000
train/	
approx_kl	0.007343081
clip_fraction	0.0368
clip_range	0.2
entropy_loss	-0.968
explained_variance	0.499
learning_rate	0.0003
loss	0.501
n_updates	11600
policy_gradient_loss	-0.00472
value_loss	1.02

Jak ukazuje tabulka 3.2, výsledky u první plně implementované verze algoritmu se výrazně liší. Jednotlivé epizody trvají déle než u první verze, protože agent musí nejen dojít k cíli, ale zároveň najít nepřítele, otočit se na něj, zahájit útok a eliminovat ho. Délka tréninku se proto přibližně ztrojnásobuje, aby agent dosáhl spolehlivého chování.

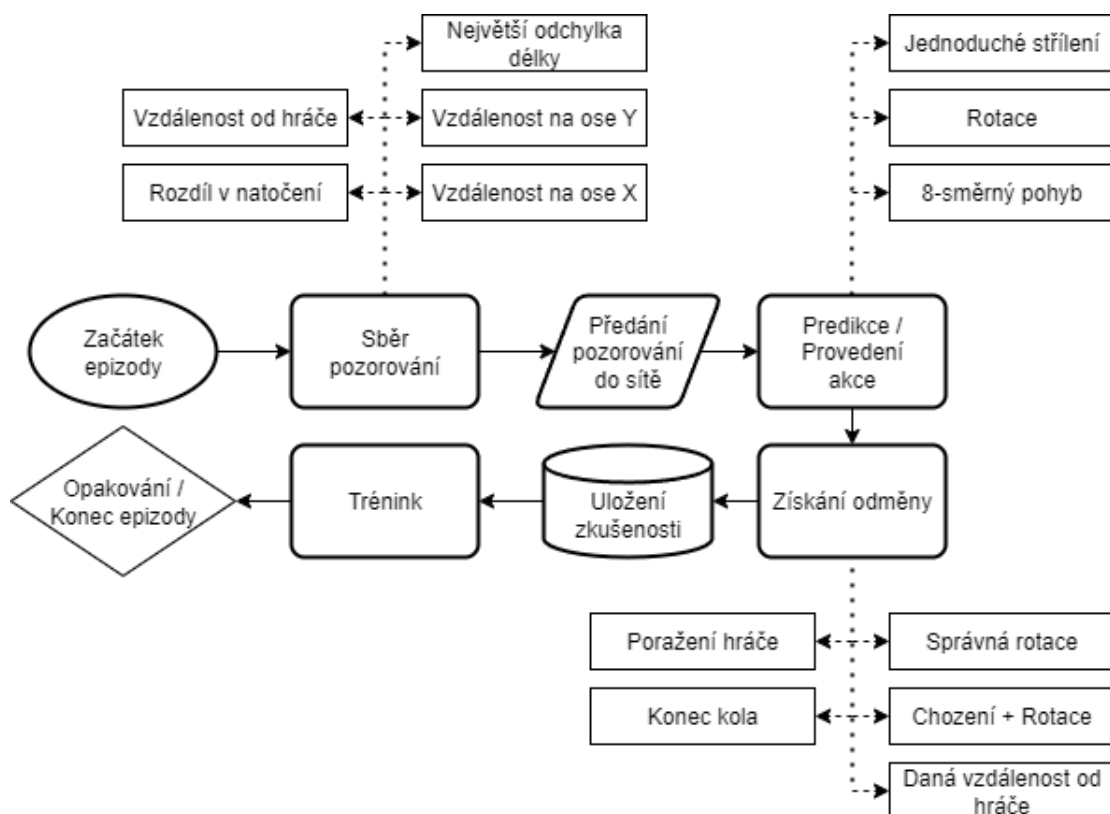
Ztráta podle policy gradientu se zvyšuje z -0.00192 na -0.00472 , což ukazuje na silnější gradientové signály a výraznější změny v politice agenta. Přibližná KL divergence narůstá z 0.00098 na 0.00734 , což znamená, že se politika během tréninku aktualizuje agresivněji — typický jev při zpřesňování chování.

Entropy loss klesá z -0.157 na -0.968 , což značí výrazné snížení náhodnosti v rozhodování. Agentovo chování se stává více deterministickým, což často signalizuje pozitivní konvergenci. Naopak hodnota explained variance klesá z 0.926 na 0.499 , což znamená, že přesnost predikcí hodnotové funkce (kritiky) se zhoršuje. Predikce budoucích odměn jsou méně spolehlivé — běžný jev v hlubokém zpětnovazebním učení, obzvláště při nasazení konkrétních strategií.

Pozoruhodným rozdílem je také nárůst value loss z 0.00187 na 1.02 , což může indikovat určitou nestabilitu hodnotové funkce během tréninku.

Bohužel, i když se tento model dokáže poměrně dobře adaptovat na hráče, stále vyžaduje úpravy. Ignoruje některá pravidla odměňování — místo aby si udržoval optimální vzdálenost od nepřítele, přibližuje se co nejbližší, aby maximalizoval šanci na úspěšný zásah. Děje se tak proto, že odměna za zásah výrazně převyšuje penalizaci za nevhodnou vzdálenost. Kromě toho dosud chybí implementace poslední požadované funkcionality.

Vývojový diagram na obrázku 3.3 znázorňuje strukturu algoritmu agenta odpovídající této konkrétní verzi.



Obrázek 3.3: Vývojový diagram algoritmu agenta – druhá verze

3.2.3 Finální verze agenta

Modifikace prostředí

Jednoduchého agenta ze základního tréninku nahrazuje agent označovaný jako NPC, který využívá propracovanější střeleckou logiku přímo implementovanou na zbraních navržených speciálně pro něj. Odměna za zásah je snížena, což zabraňuje agentovi získávat body tím, že se přiblíží k hráči na extrémně krátkou vzdálenost a opakovaně jej zasahuje. NPC agent má schopnost volby mezi třemi typy zbraní, čímž získává vyšší funkční flexibilitu.

Dále dokáže přijímat poškození — pokud mu klesne život na nulu, umírá a na jeho místě se objeví lékárnička a munice, jak bylo popsáno výše. Před každým novým kolem si agent zapamatuje zbraň a brnění, které hráč použil v předchozím kole, a na základě těchto informací volí optimální kombinaci výbavy pro kolo následující.

V odměnové logice je přidáno nové omezení a je upraveno jedno existující:

1. Kombinovaná odměna:

- Pokud jsou současně splněny podmínky správného natočení a optimální vzdálenosti od cíle, agent získá odměnu $+0,1$. V opačném případě obdrží penalizaci $-0,1$.

2. Eliminace nepřítele:

- Pokud agent úspěšně eliminujete nepřítele, epizoda končí a připisuje se hlavní odměna. Výchozí hodnota činí $5,0$, ale násobí se koeficientem závislým na aktuální vzdálenosti v okamžiku zásahu – nejvyšší odměna je udělena při optimálním odstupu od cíle.

Finální verze modelu

Tabulka 3.3: Výsledky základního tréninku finální verze agenta

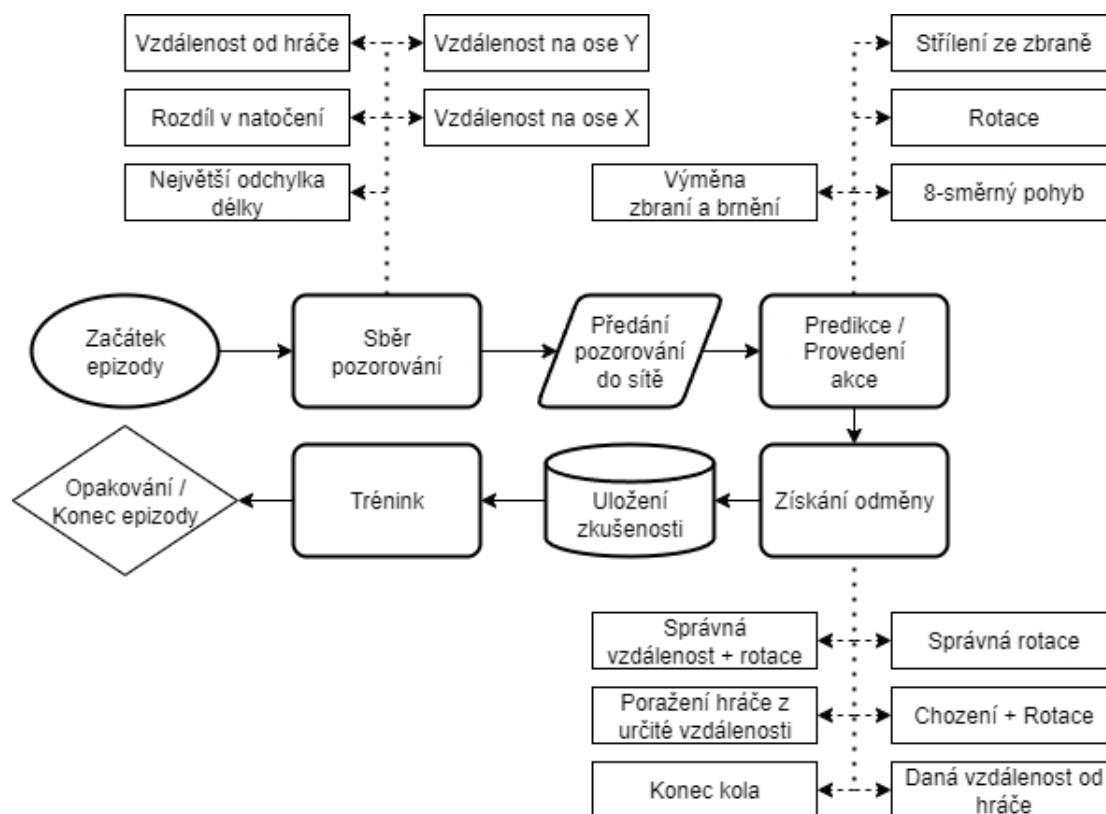
Metrika	Hodnota
rollout/	
ep_len_mean	27.4
ep_rew_mean	8.8
time/	
fps	2500
iterations	59
time_elapsed	1237
total_timesteps	3240000
train/	
approx_kl	0.004143081
clip_fraction	0.0332
clip_range	0.2
entropy_loss	-0.994
explained_variance	0.604
learning_rate	0.0003
loss	0.301
n_updates	11600
policy_gradient_loss	-0.00432
value_loss	0.7

Tabulka 3.3 ukazuje výsledky finální verze modelu, která vykazuje zlepšení ve všech sledovaných oblastech oproti verzi druhé. Jednotlivé epizody trvají mírně kratší dobu — agent nyní nachází nepřítele rychleji, otáčí se směrem k němu, zahajuje útok a eliminuje ho, přičemž tréninková doba zůstává zachována.

„Ztráta podle policy gradientu“ klesá z $-0,00472$ na $-0,00432$, což naznačuje menší výkyvy v aktualizaci politiky — žádoucí trend. „Přibližná KL divergence“ se snižuje z $0,00734$ na $0,00414$, což znamená, že aktualizace politiky probíhají méně agresivně — agentovo chování je stabilnější a přesnější. „Entropy loss“ klesá z $-0,968$ na $-0,994$, což odráží další redukci náhodnosti v rozhodování. „Explained variance“ roste z $0,499$ na $0,604$, čímž se zlepšuje schopnost modelu odhadovat hodnotu stavu a predikovat budoucí odměny. „Value loss“ se snižuje z $1,02$ na $0,70$, což poukazuje na nižší nestabilitu hodnotové funkce.

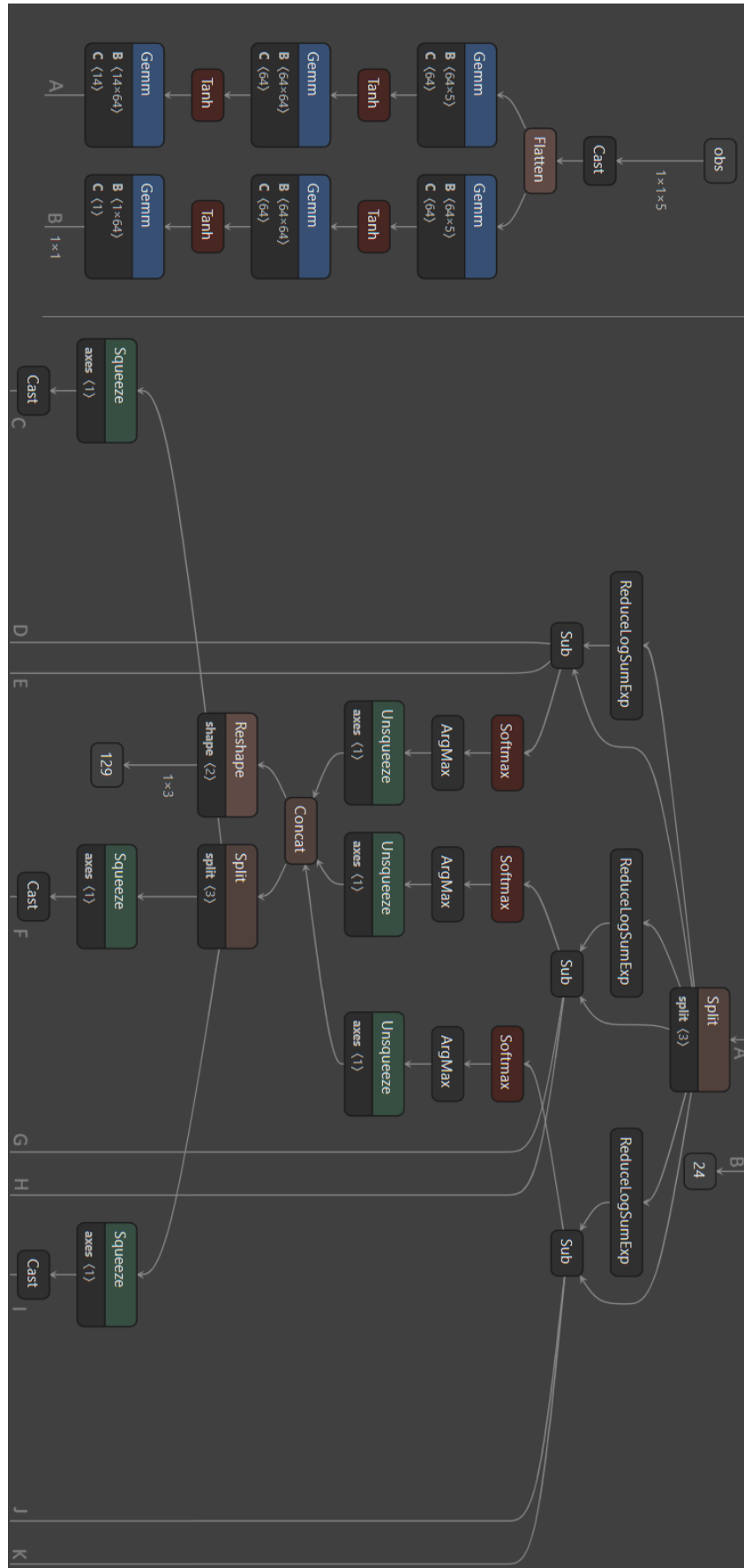
Finální agent dokáže správně volit zbraň a brnění, pronásledovat hráče, nasměrovat se k němu a zahájit střelbu. Tyto volby navíc dokáže během činnosti velice pomalu přizpůsobovat a optimalizovat podle aktuální situace. Přesto systém není zcela bezchybný — při pohybu hráče občas agent místo jemné korekce rotace provede úplný obrat, než se správně natočí.

Obrázek 3.4 představuje vizuální přehled rozhodovací logiky agenta, jak byla implementována v rámci této verze.

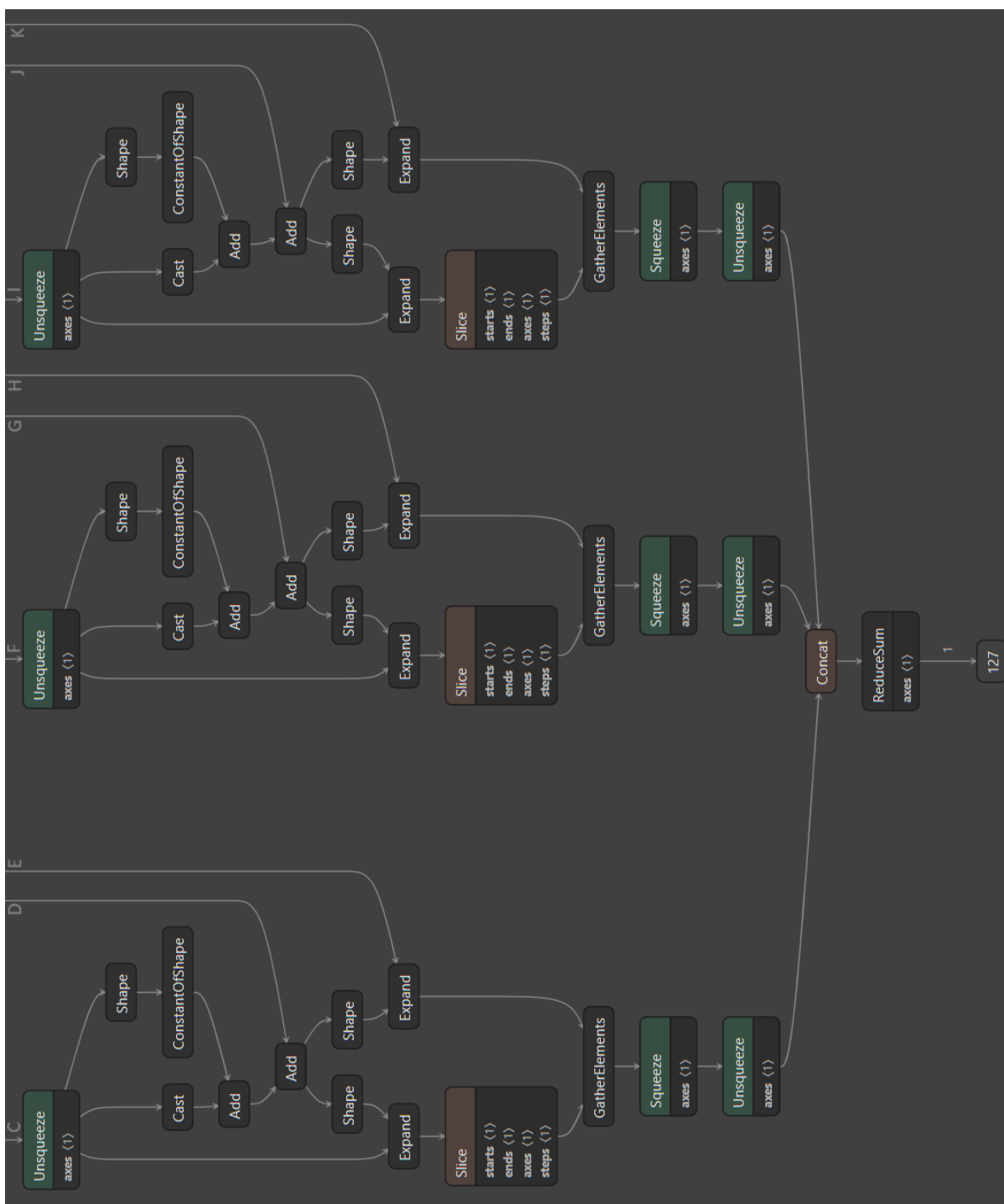


Obrázek 3.4: Vývojový diagram algoritmu agenta – finální verze

Obrázky 3.5 a 3.6 ilustrují vizuální podobu finálního modelu.



Obrázek 3.5: Vzhled modelu z finální verze agenta 1/2



Obrázek 3.6: Vzhled modelu z finální verze agenta 2/2

3.2.4 Shrnutí všech verzí

První verze agenta se sice dokáže adaptovat a funguje nejlépe, ale zároveň je nejjednodušší, protože zvládá pouze pohyb. Hráč proto tento model dokáže snadno porazit. Druhá verze obsahuje veškerou potřebnou logiku, ale nefunguje a neadaptuje se dle očekávání — agent se neustále drží těsně u nepřítele, což sice může vést k vítězství, ale působí nepřírozeně. Finální verze je ze všech nejvyspělejší — učí se, adaptuje se a funguje téměř bezchybně: skutečně pronásleduje hráče, mění výbavu, útočí apod. Tato verze je zároveň nejobtížnější na poražení, což byl zamýšlený cíl.

Bohužel, přestože se agent během hry dále učí a adaptuje na hráče, ve hře se vyskytuje výrazně méně instancí agentů než na trénovací mapě, takže online trénink během hraní je téměř zanedbatelný.

Závěr

Hlavním cílem této diplomové práce bylo navrhnout jednoduchého autonomního agenta, který se dokáže za pomoci strojového učení a změny vybavení adaptovat v dynamickém prostředí a snadněji porazit protihráče.

Nejprve jsem popsal existující AI agenty ve hrách a nejčastěji používané techniky a algoritmy. Mezi zmíněnými agenty byli AlphaStar, OpenAI Five a Voyager; z algoritmů a technik pak PPO, Deep Q-Network, Self-Play a Population-Based Training.

V další části jsem uvedl technologie použité při tvorbě samotné hry a agenta — např. Unreal Engine či plugin NevarokML. Popsán byl také návrh hry i samotného agenta.

Závěrečná část práce se věnovala implementaci jednotlivých modulů hry, jako jsou hráčská postava, zbraně, systém kol, získávání vybavení (zbraní a brnění) a především implementaci, trénování a aplikaci agenta. Tento agent využívá logiku vytvořenou pomocí strojového učení a je schopen pohybu, rotace, útoku a změny výbavy.

První verze agenta zvládala pouze základní pohyb a jejím hlavním účelem bylo osvojení si práce s pluginem NevarokML. Druhá verze, která představovala první plně funkční implementaci, vykazovala částečnou úspěšnost — agent se občas zcela zasekl a nevěděl, jak reagovat, nebo se místo udržování odstupu příliš přibližoval k nepříteli. Finální verze agenta, navzdory několika drobným nedostatkům, nejlépe prokázala schopnost adaptace v dynamickém prostředí a na chování hráče. Dokáže se pohybovat, otáčet, útočit a po každém kole měnit svou výbavu.

Literatura

- [1] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2025-05-10].
- [2] CONTRIBUTORS, Wikipedia. *ChatGPT*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-, https://en.wikipedia.org/wiki/Main_Page. Dostupné z: <https://en.wikipedia.org/w/index.php?title=ChatGPT&oldid=1290307244>. [cit. 2025-05-10].
- [3] CONTRIBUTORS, Wikipedia. *Dota 2*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=Dota_2&oldid=1288870722. [cit. 2025-05-10].
- [4] JADERBERG, Max; DALIBARD, Valentin a OSINDERO, Simon et al. *Population Based Training of Neural Networks*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1711.09846>. [cit. 2025-05-10].
- [5] *MuJoCo*. Online. C2021. Dostupné z: <https://mujoco.org/>. [cit. 2025-05-10].
- [6] TEAM, OpenAI Five. *OpenAI Five defeats Dota 2 world champions*. Online. OpenAI. 2019. Dostupné z: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/#timeline>. [cit. 2025-05-10].
- [7] *Proximal Policy Optimization*. Online. OpenAI. 2017. Dostupné z: <https://openai.com/index/openai-baselines-ppo/>. [cit. 2025-05-10].
- [8] SCHULMAN, John; LEVINE, Sergey; MORITZ, Philipp; JORDAN, Michael I. a ABBEEL, Pieter. *Trust Region Policy Optimization*. Online, arxiv. USA: Cornell University, 2015. Dostupné z: <http://arxiv.org/abs/1502.05477>. [cit. 2025-05-10].
- [9] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec a KLIMOV, Oleg. *Proximal Policy Optimization Algorithms*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1707.06347>. [cit. 2025-05-10].
- [10] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael a ABBEEL, Pieter. *High-Dimensional Continuous Control Using Generalized*

- Advantage Estimation*. Online, arxiv. USA: Cornell University, 2018. Dostupné z: <https://arxiv.org/abs/1506.02438>. [cit. 2025-05-10].
- [11] VINYALS, Oriol; BABUSCHKIN, Igor; CHUNG, Junyoung et al. *AlphaStar: Mastering the real-time strategy game StarCraft II*. Online. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-05-10].
- [12] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. Online. MineDojo. B. r. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-05-10].
- [13] WATKINS, Christopher J. C. H. a DAYAN, Peter. Q-learning. Online. *Machine Learning*. 1992, roč. 8, č. 11, s. 279—292. Dostupné z: <https://doi.org/10.1007/BF00992698>. [cit. 2025-05-10].
- [14] CONTRIBUTORS, Wikipedia. *Deep Blue (chess computer)*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: [https://en.wikipedia.org/w/index.php?title=Deep_Blue_\(chess_computer\)&oldid=1288103584](https://en.wikipedia.org/w/index.php?title=Deep_Blue_(chess_computer)&oldid=1288103584). [cit. 2025-05-10].
- [15] CONTRIBUTORS, Wikipedia. *StarCraft II*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=StarCraft_II&oldid=1286300229. [cit. 2025-05-10].
- [16] ZHANG, Ruize; XU, Zelai a MA, Chengdong et al. *A Survey on Self-play Methods in Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2024. Dostupné z: <https://arxiv.org/abs/2408.01072>. [cit. 2025-05-10].
- [17] *Unreal Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: www.unrealengine.com
- [18] *NevarokML* [online]. China: Kyrylo Mishakin, 2025 [cit. 2025-07-31]. Dostupné z: <https://nevarok.com/nevarok-ml/>
- [19] *ONNX* [online]. USA: Microsoft, AMD et., 2025 [cit. 2025-07-31]. Dostupné z: www.onnx.ai
- [20] *Neural Network Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/neural-network-engine-overview-in-unreal-engine>
- [21] SUTTON, Richard S. a BARTO, Andrew G. *Reinforcement learning: an introduction*. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, [2018]. ISBN 978-0-262-03924-6.
- [22] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. *Playing Atari with Deep Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2013. Dostupné z: <https://arxiv.org/abs/1312.5602>. [cit. 2025-07-31].

- [23] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. Human-level control through deep reinforcement learning. Online. *Nature*. 2015, č. 518, s. 529—533. Dostupné z: <https://doi.org/10.1038/nature14236>. [cit. 2025-07-31].
- [24] TEAM, AlphaStar. *AlphaStar: Mastering the real-time strategy game StarCraft II*. 2019. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-07-31].
- [25] TEAM, OpenAI Five. *OpenAI Five*. 2018. Dostupné z: <https://openai.com/index/openai-five/>. [cit. 2025-07-31].
- [26] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-07-31].
- [27] PATHMAKUMAR, Thejus et al. *A Reinforcement Learning Based Dirt-Exploration for Cleaning-Auditing Robot*. 2021. Dostupné z: https://www.researchgate.net/figure/The-reinforcement-learning-architecture-based-on-Proximal-Policy-Optimization-fig3_357019705. [cit. 2025-07-31].
- [28] STROUSE, DJ et al. *Collaborating with Humans without Human Data*. Online, arxiv. USA: Cornell University, 2021. Dostupné z: <https://arxiv.org/abs/2110.08176>. [cit. 2025-07-31].
- [29] JADERBERG, Max. *Population based training of neural networks*. 2017. Dostupné z: <https://deepmind.google/discover/blog/population-based-training-of-neural-networks/>. [cit. 2025-07-31].
- [30] ELSAYED, Medhat et al. *AI-enabled Future Wireless Networks: Challenges, Opportunities and Open Issues*. 2021. Dostupné z: https://www.researchgate.net/publication/349913716_AI-enabled_Future_Wireless_Networks_Challenges_Opportunities_and_Open_Issues. [cit. 2025-07-31].

Příloha A

Jak zprovoznit program

1. Nejprve si stáhněte soubor přiložený k diplomové práci z onedrive, který obsahuje samotný projekt a také pluginy, jež je nutné přidat do Unreal Engine.
2. Pokud nemáte nainstalovaný „Epic Games Store“, stáhněte si jej ze stránky <https://store.epicgames.com/>. V pravém horním rohu klikněte na tlačítko „Download“ a spusťte instalátor, čímž nainstalujete „Epic Games Launcher“.
3. Po spuštění aplikace si vytvořte účet (pokud jej ještě nemáte), buď přímo v „Epic Games Store“, nebo prostřednictvím „Epic Games Launcher“, a přihlaste se.
4. Po přihlášení klikněte v levém menu na položku „Unreal Engine“.
5. V horní části aplikace se zobrazí nové možnosti — vyberte záložku „Library“.
6. V sekci „Engine Versions“ klikněte na tlačítko „+“. Objeví se nabídka výběru verze engine. Zvolte verzi 5.2.1 a spusťte instalaci. Umístění instalace můžete ponechat výchozí, ale zapamatujte si ho — bude potřeba později.
7. Po dokončení instalace přejděte do složky s nainstalovaným enginem, která se bude jmenovat např. UE_5.2. Do této složky vložte podsložku **Engine** ze složky „Plugins“, která je umístěna vedle složky projektu „RoboHell“. Tímto krokem zajistíte správné nainstalování potřebných pluginů.
8. Projekt spusťte dvojklikem na soubor „RoboHell.uproject“ ve složce projektu. První spuštění může trvat déle, protože probíhá příprava engine a kompilace projektu.
9. Po načtení projektu jej zatím nespouštějte pomocí zeleného trojúhelníku! Před prvním spuštěním je třeba provést ještě několik kroků.
10. V levém horním rohu klikněte na „Edit“ a v rozbaleném menu vyberte „Editor Preferences“.
11. V zobrazeném nastavení sjedťte v levém panelu dolů k sekci „Plugins“ a rozklikněte „NevarokML“.

12. Klikněte na tlačítko „Refresh Details“. Vizuálně se nic nezmění, ale tímto krokem se plugin správně načte v editoru.
13. Tímto by mělo být vše připraveno. Projekt by nyní měl být plně funkční. Doporučuji však přecíst také kapitolu „Jak opravit problém s projektem“, která řeší častou chybu při spouštění.

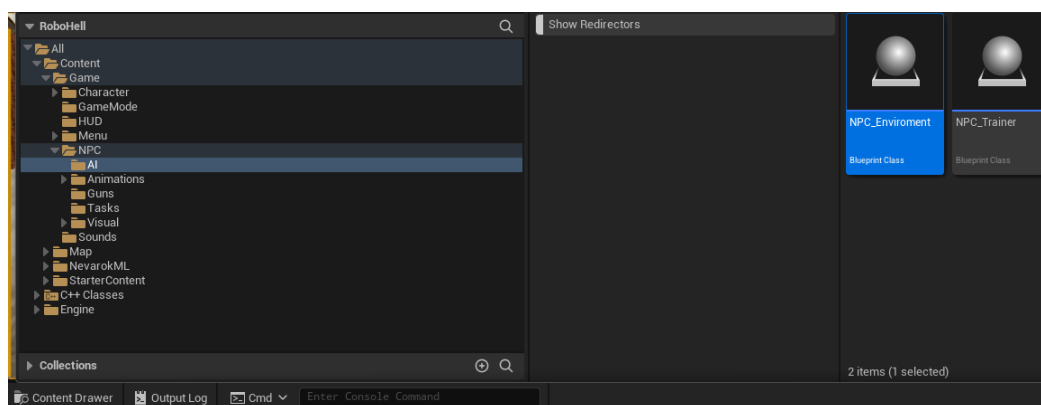
Příloha B

Jak opravit problém s programem

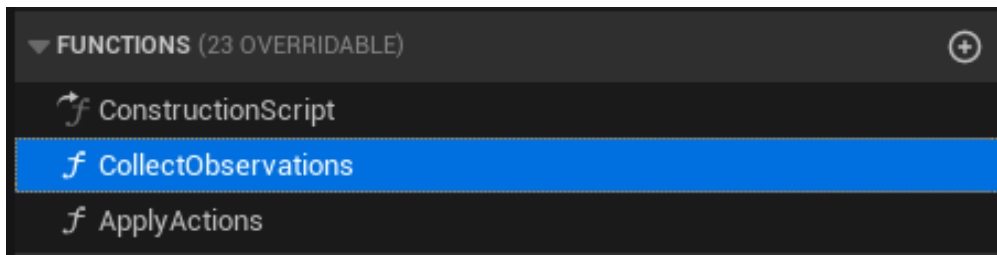
1. Pokud se zdá, že agent hráče nenásleduje a odchází jedním směrem pryč, nebo se na obrazovce zobrazí chybová hláška znázorněná na obrázku 1, tato příloha vám poradí, jak danou chybu opravit.

```
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-4) < 0 || x(-4) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-4) < 0 || x(-4) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
```

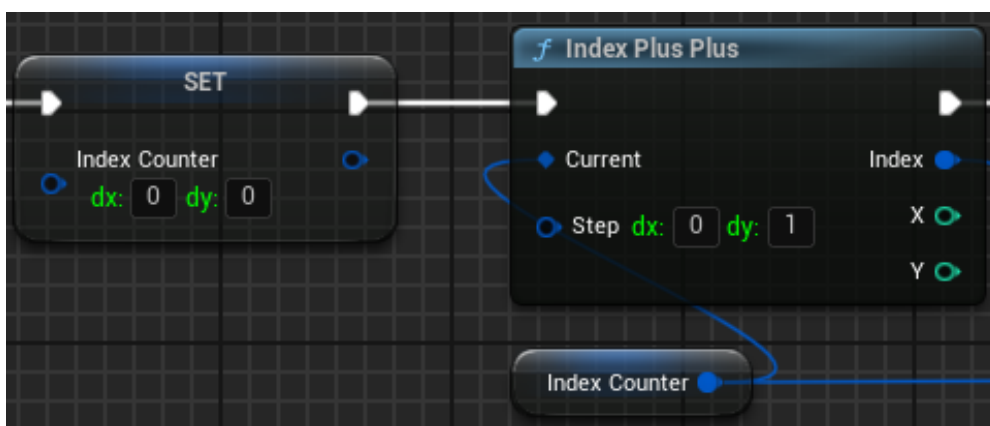
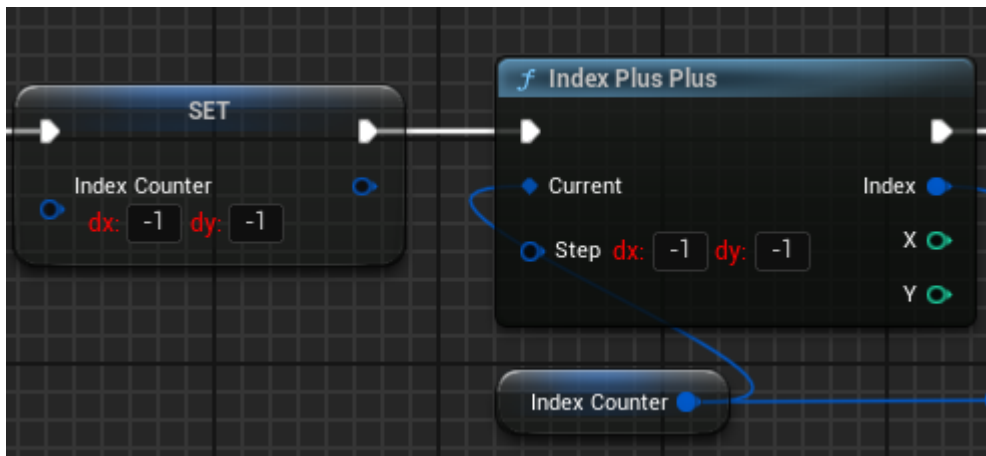
2. Tato chyba je bohužel známým problémem pluginu NevarokML, ale naštěstí má jednoduché řešení.
3. Po otevření projektu klikněte ve spodní části editoru na tlačítko „Content Drawer“ a přejděte do složky All/Content/NPC/AI, jak je znázorněno na obrázku 4.
4. Otevřete „Blueprint“ s názvem NPC_Environment.



5. Po otevření „Blueprintu“ byste měli na levé straně vidět sekci „Functions“. Zde otevřete funkci „CollectObservations“, jak je znázorněno na obrázku 5. Pokud tuto sekci nevidíte, klikněte nahoře na položku „Window“ a v části „Graph Editor“ zaškrtněte možnost „My Blueprint“.



6. Ve funkci „CollectObservations“ se nachází uzel „Set Index Counter“ a pět uzlů s názvem „Index Plus Plus“. Pokud se agent nepohybuje nebo se zobrazila chyba z obrázku 1, je pravděpodobné, že tyto uzly mají nesprávné hodnoty, jak je vidět na obrázku 7.
7. V uzlu „Set Index Counter“ nahraďte obě červeně označené hodnoty -1 hodnotou 0. U všech pěti uzlů „Index Plus Plus“ pak změňte hodnoty tak, aby první pole obsahovalo 0 a druhé 1, jak ukazuje obrázek 7.



8. Tímto je oprava dokončena a agent by nyní měl fungovat správně.